KENNESAW STATE
UNIVERSITY

CS 4277
Image Segmentation-DL

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

Lecture Overview

- Computer Vision and Image Segmentation
- Semantic Segmentation: Sliding window/convolution/fully convolution
- Key techniques in fully convolutional network
  - Upsampling/unpooling
  - Convolution transpose
- Morden Image Segmentation Neural Network
  - Fully convolution network
  - Encoder-decoder based network
  - Multi-scale and Pyramid Network
  - Dilated convolutional models and deeplab family
  - Attention based models

Computer Vision and Image segmentation

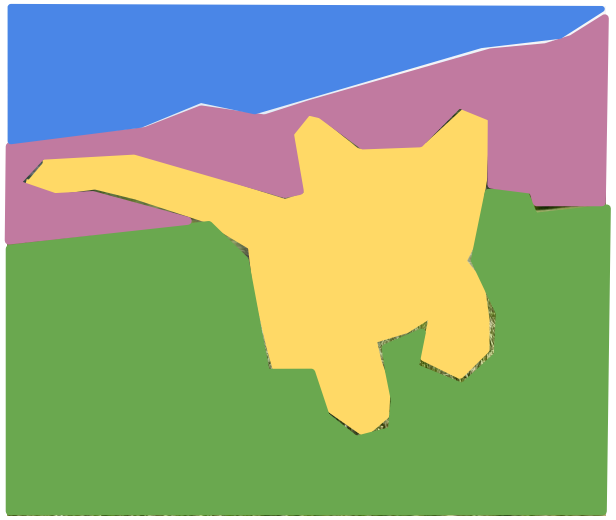
Computer Vision Tasks

Classification

CAT

No spatial extent

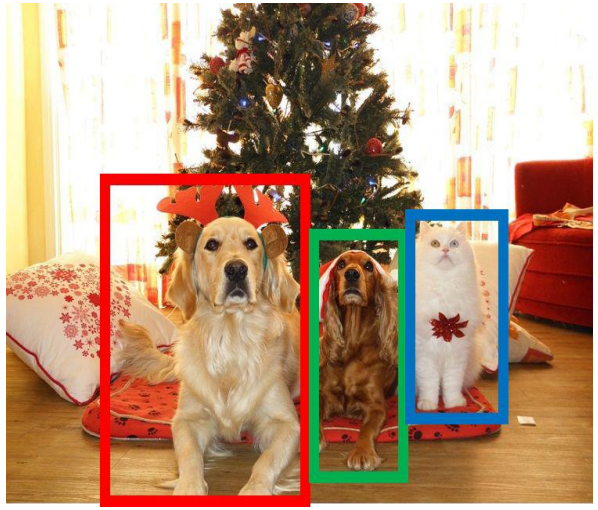
Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Instance Segmentation

DOG, DOG, CAT

Multiple Object

This image is CC0 public domain

Image Classification: A core task in Computer Vision

(assume given a set of possible labels)
{dog, cat, truck, plane, ...}

→ cat

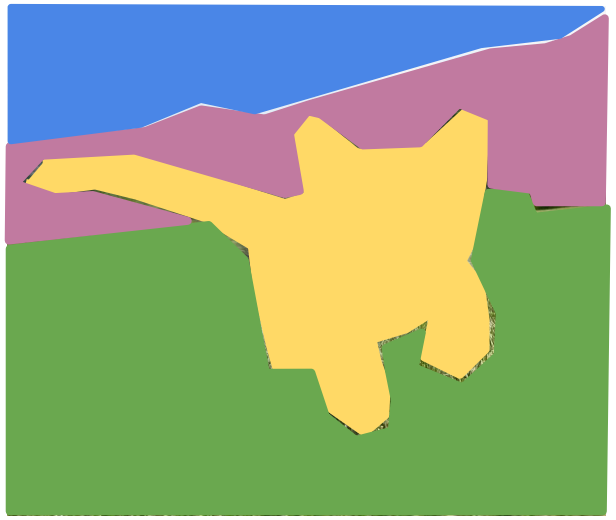
Semantic Segmentation

Classification

CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection

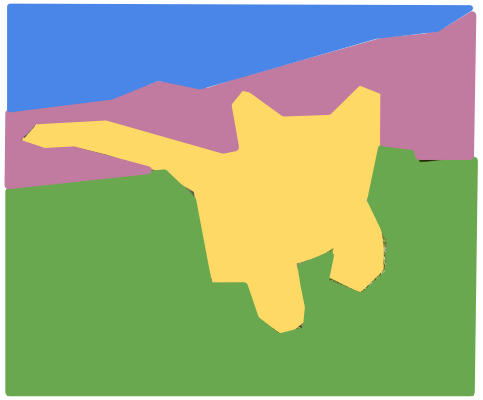
DOG, DOG, CAT

Instance Segmentation

DOG, DOG, CAT

Multiple Object

Semantic Segmentation: The Problem



GRASS, CAT,
TREE, SKY, ...

Paired training data: for each training image, each pixel is labeled with a semantic category.

?

At test time, classify each pixel of a new image.

Semantic Segmentation Idea: Sliding Window

Full image

Semantic Segmentation Idea: Sliding Window

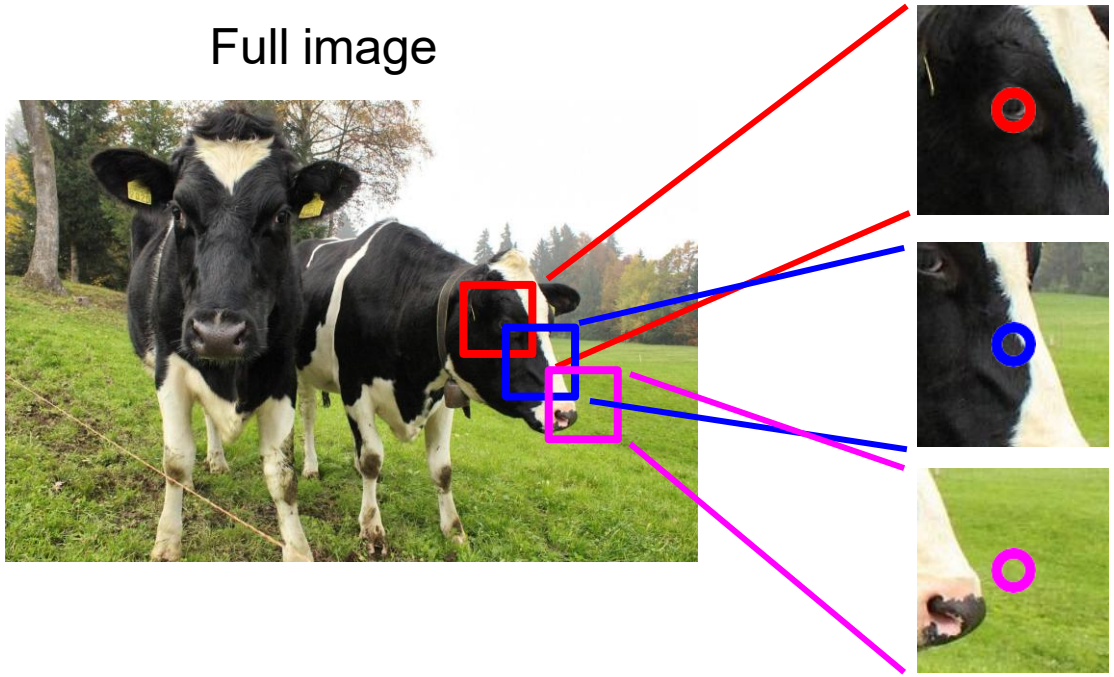
Full image

?

Impossible to classify without context

Q: how do we include context?

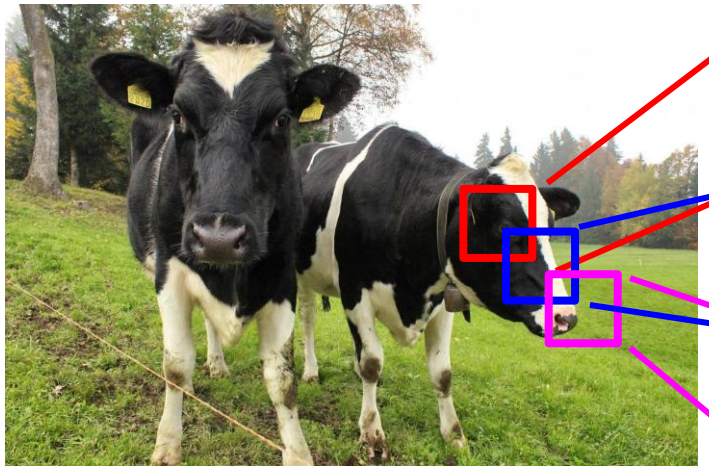
Semantic Segmentation Idea: Sliding Window



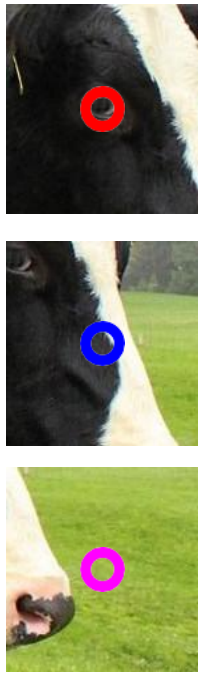
Q: how do we model this?

Semantic Segmentation Idea: Sliding Window

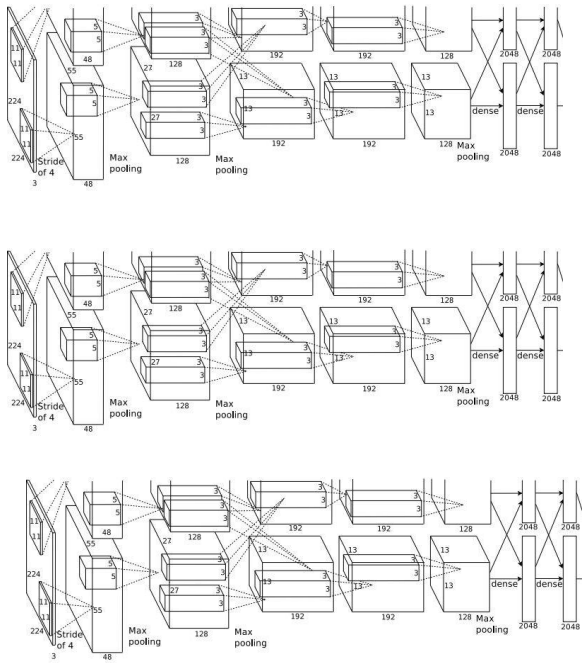
Full image



Extract patch



Classify center pixel with CNN



Cow

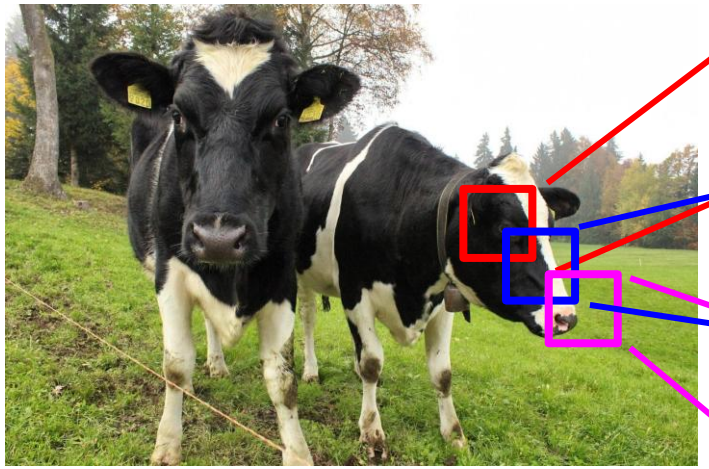
Cow

Grass

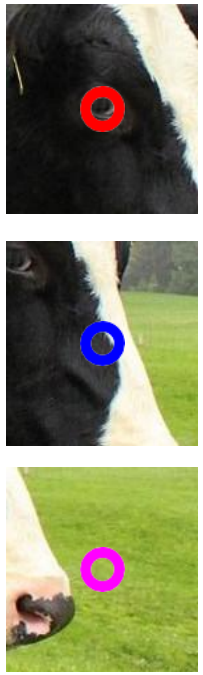
Farabet et al, “Learning Hierarchical Features for Scene Labeling,” TPAMI 2013
Pinheiro and Collobert, “Recurrent Convolutional Neural Networks for Scene Labeling”, ICML 2014

Semantic Segmentation Idea: Sliding Window

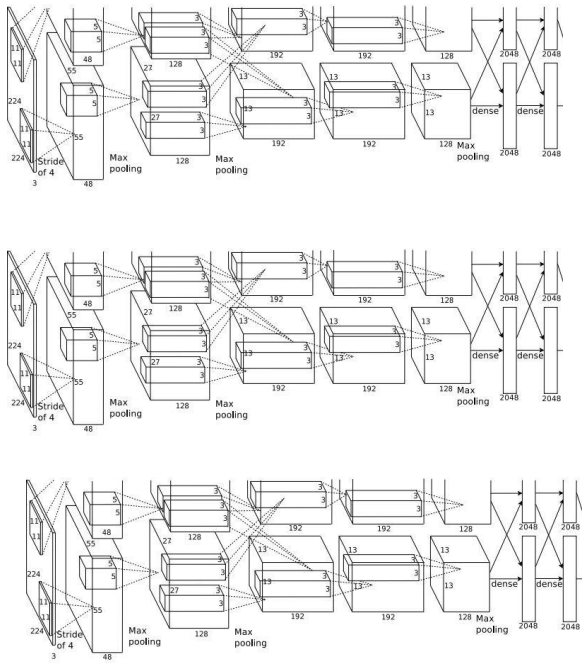
Full image



Extract patch



Classify center pixel with CNN



Cow

Cow

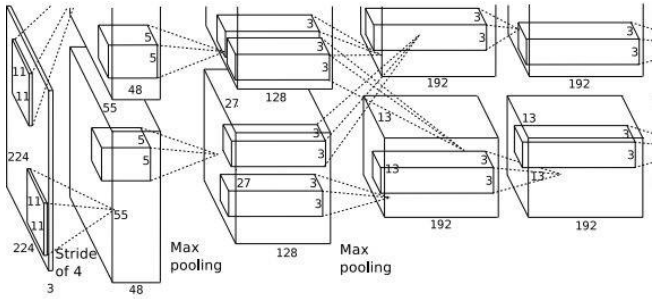
Grass

Problem: Very inefficient! Not reusing shared features between overlapping patches

Farabet et al, “Learning Hierarchical Features for Scene Labeling,” TPAMI 2013
Pinheiro and Collobert, “Recurrent Convolutional Neural Networks for Scene Labeling”, ICML 2014

Semantic Segmentation Idea: Convolution

Full image

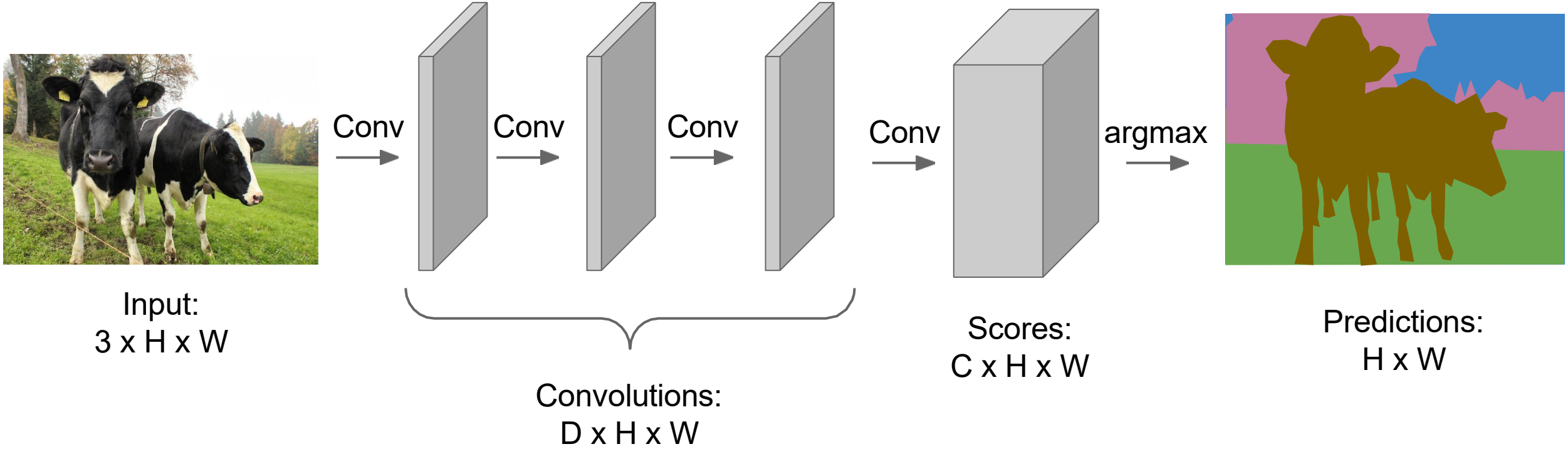


An intuitive idea: encode the entire image with conv net, and do semantic segmentation on top.

Problem: classification architectures often reduce feature spatial sizes to go deeper, but semantic segmentation requires the output size to be the same as input size.

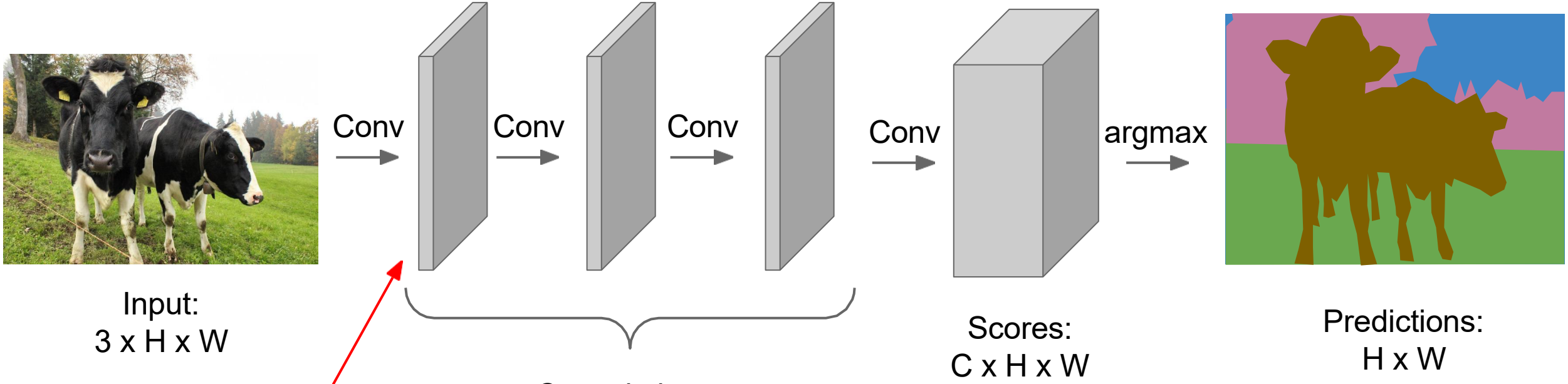
Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!



Semantic Segmentation Idea: Fully Convolutional

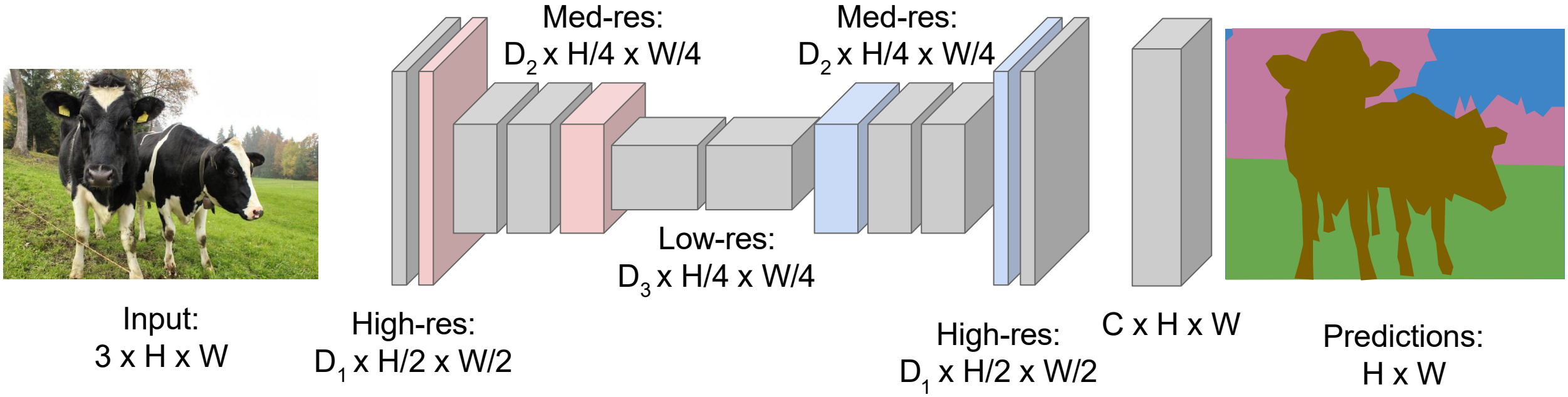
Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!



Problem: convolutions at original image resolution will be very expensive ...

Semantic Segmentation Idea: Fully Convolutional

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Long, Shelhamer, and Darrell, “Fully Convolutional Networks for Semantic Segmentation”, CVPR 2015
Noh et al, “Learning Deconvolution Network for Semantic Segmentation”, ICCV 2015

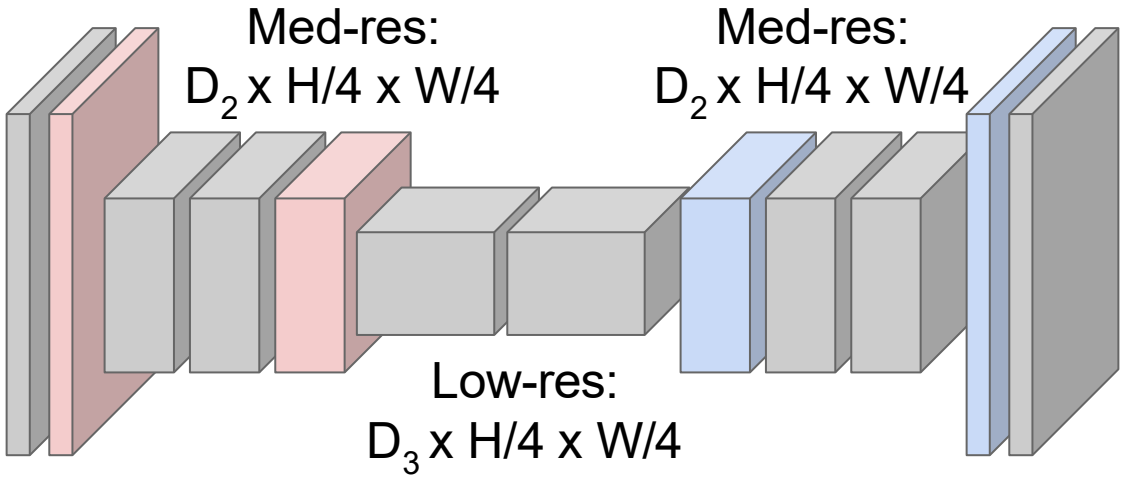
Semantic Segmentation Idea: Fully Convolutional

Downsampling: Pooling, strided convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

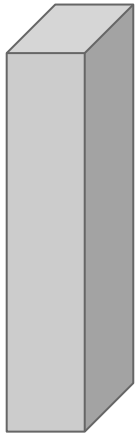
Upsampling: ???

Input: 3 x H x W



High-res: D_1 x H/2 x W/2

High-res: D_1 x H/2 x W/2



C x H x W

Predictions: H x W

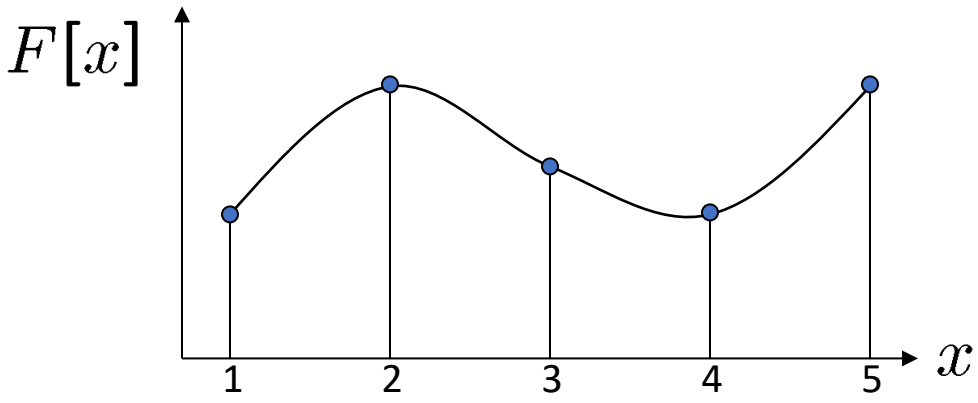
Long, Shelhamer, and Darrell, “Fully Convolutional Networks for Semantic Segmentation”, CVPR 2015
Noh et al, “Learning Deconvolution Network for Semantic Segmentation”, ICCV 2015

Upsampling - Image interpolation

Upsampling

- This image is too small for this screen:
- How can we make it 10 times as big?
- Simplest approach: repeat each row and column 10 times
- (“Nearest neighbor interpolation”)

Image interpolation



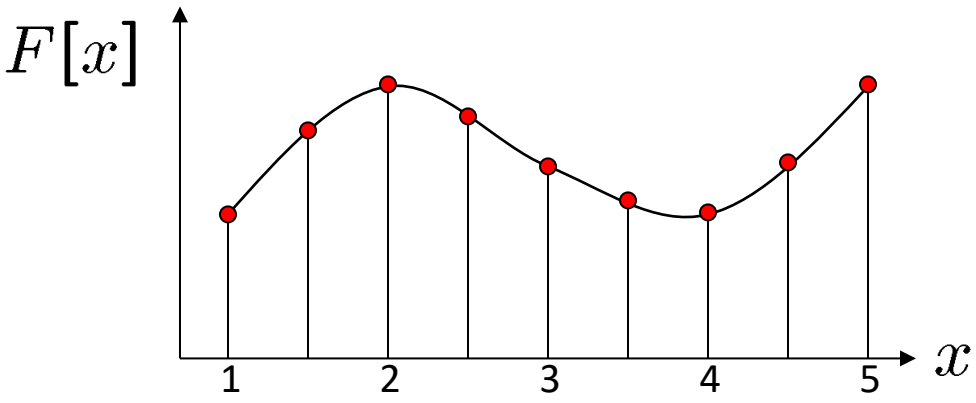
d = 1 in this example

- Recall that a digital images is formed as follows:

$$F(x, y) = quantize(f(x, y))$$

  - It is a discrete point-sampling of a continuous function
  - If we could somehow reconstruct the original function, any new image could be generated, at any resolution and scale

Image interpolation



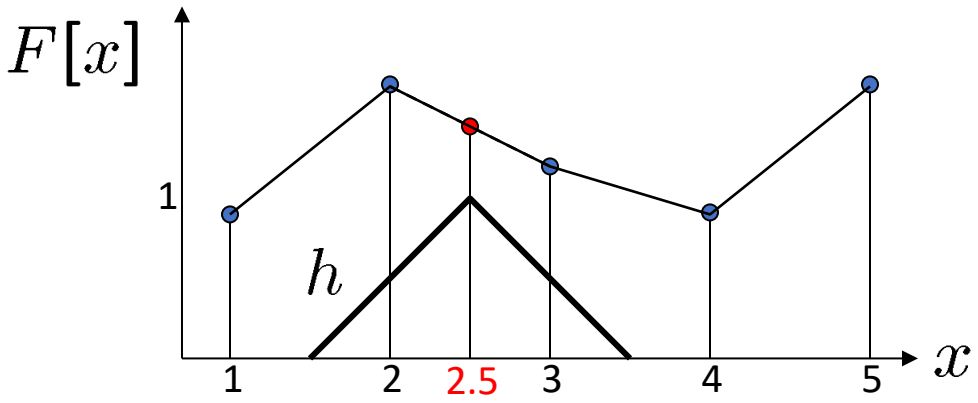
d = 1 in this example

- Recall that a digital images is formed as follows:

$$F(x, y) = quantize(f(x, y))$$

  - It is a discrete point-sampling of a continuous function
  - If we could somehow reconstruct the original function, any new image could be generated, at any resolution and scale

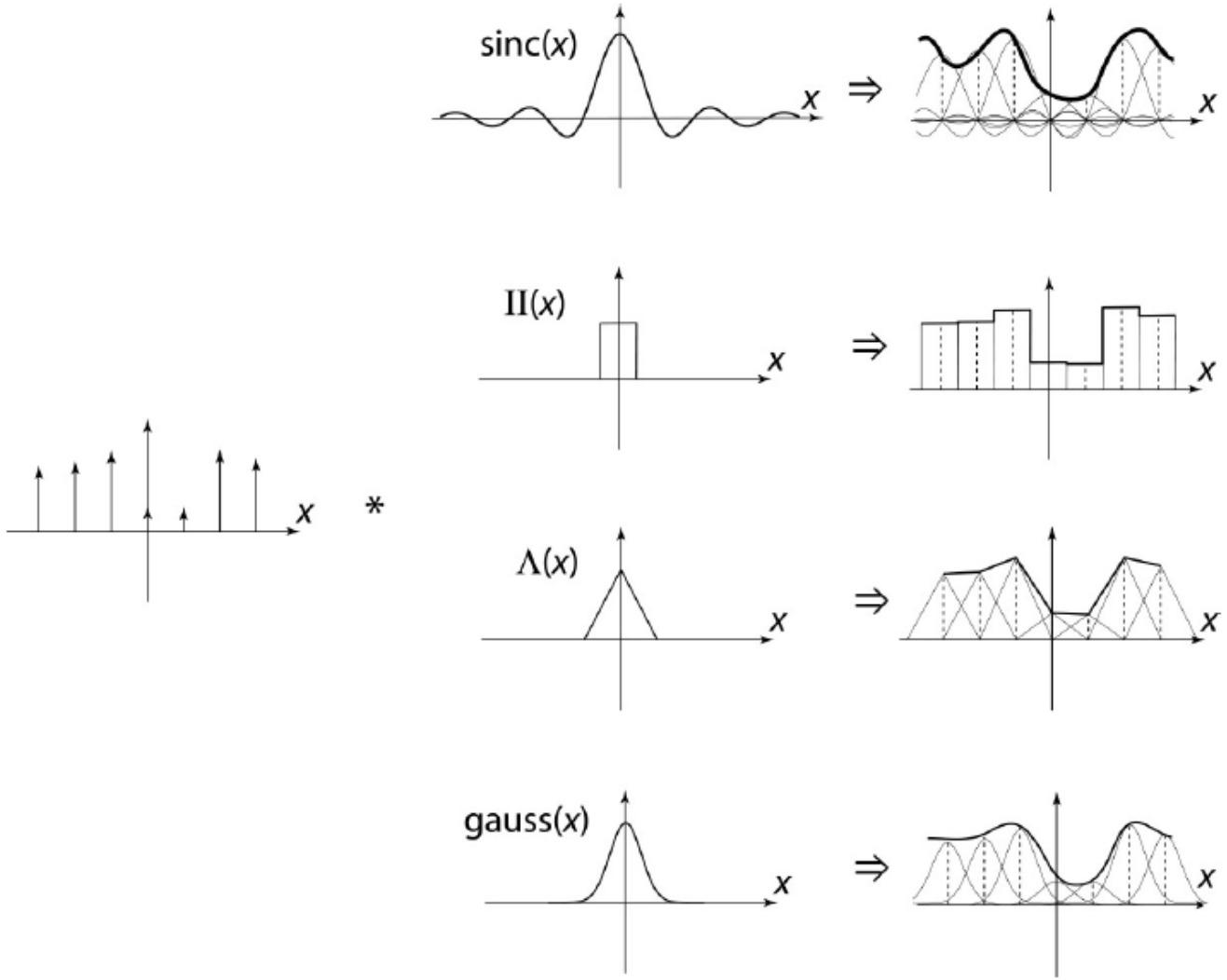
Image interpolation



d = 1 in this example

- What if we don't know f?
  - Guess an approximation: $\hat{f}$
  - Can be done in a principled way: filtering
  - Convert F to a continuous function

Image interpolation



“Ideal” reconstruction

Nearest-neighbor interpolation

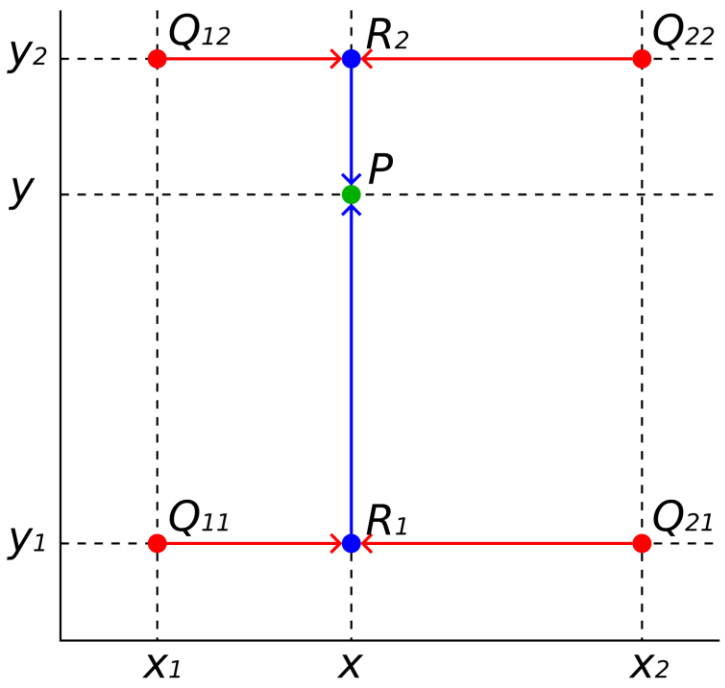
Linear interpolation

Gaussian reconstruction

Nearest neighbor interpolation

- Definition: A simple interpolation technique that selects the value of the closest data point for any given input position.
- How it Works:
  - For a target point, it identifies the nearest known data point.
  - Assigns the value of that data point to the target point without considering other surrounding points.
- Key Features
  - Fast and Simple: No complex calculations.
  - Discrete and Piecewise Constant: Results in blocky or pixelated output.
  - No Smoothing: Preserves the original sharp edges in the data.
  - Low accuracy for smooth transitions.

Bi-linear interpolation



- Linear interpolation in the x-direction

$$f(x, y_1) = \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21})$$

$$f(x, y_2) = \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22})$$

- Linear interpolation in the y-direction

$$f(x, y) = \frac{y_2 - y}{y_2 - y_1} f(x, y_1) + \frac{y - y_1}{y_2 - y_1} f(x, y_2)$$

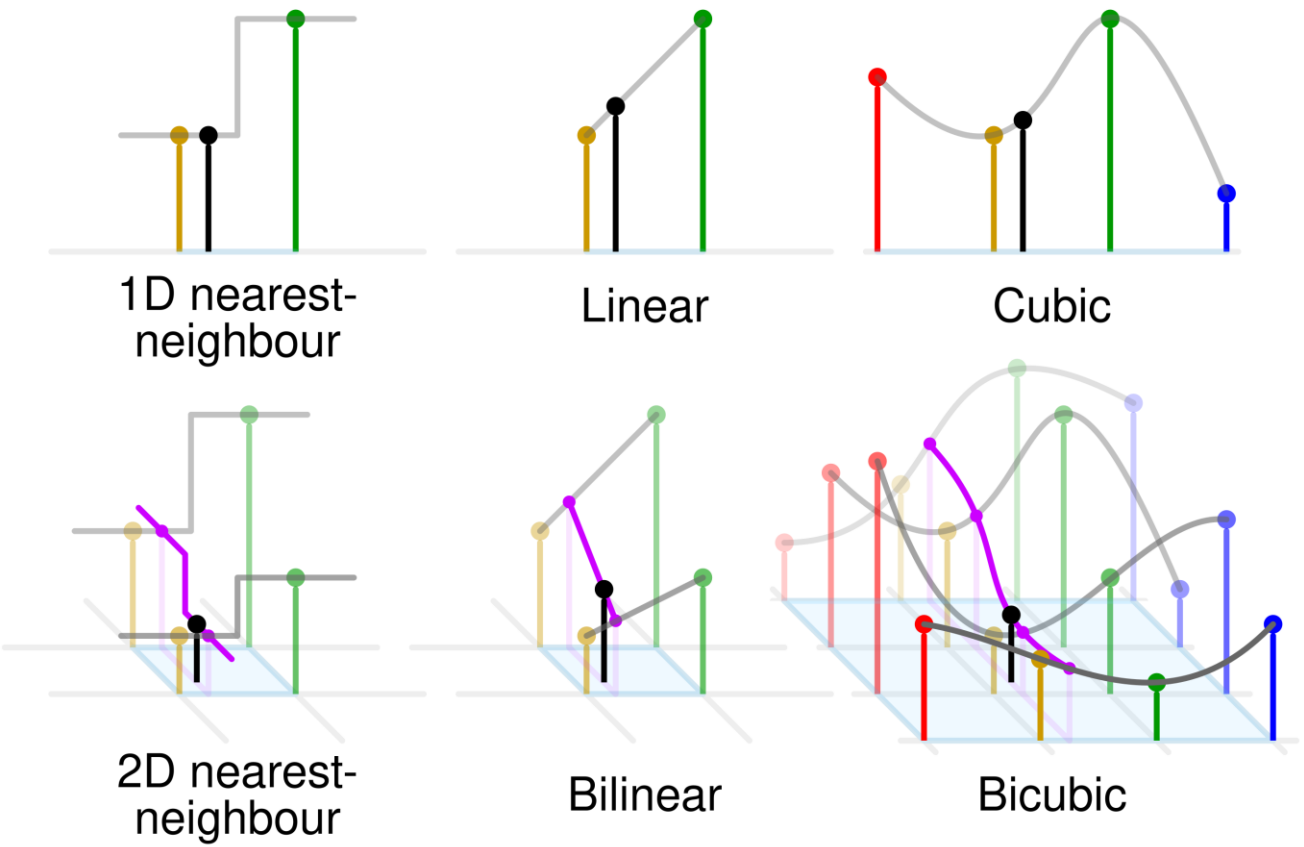
Bi-cubic interpolation

- Bi-cubic interpolation involves the sixteen nearest neighbors of a point

$$f(x,y) = \sum_{i=0}^{3}\sum_{j=0}^{3} a_{ij}x^i y^j$$

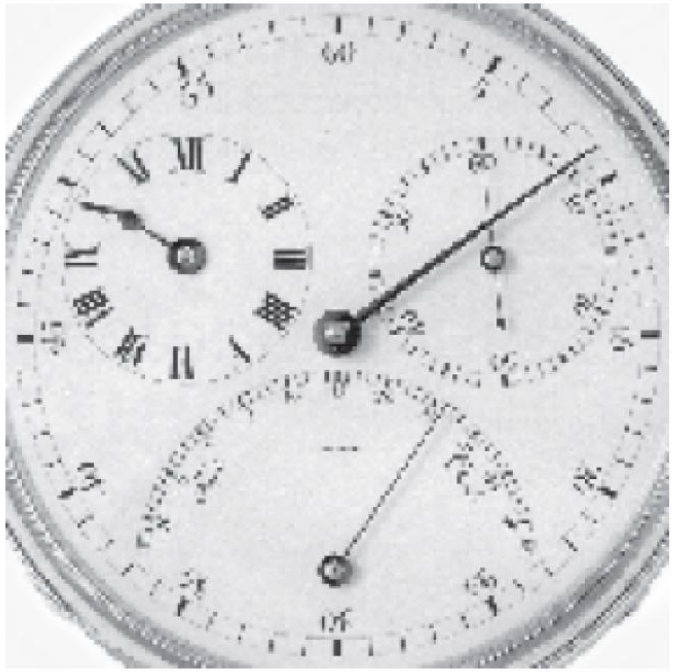
- a_{ij} is determine by 16 equations.
- https://en.wikipedia.org/wiki/Bicubic_interpolation

Comparison



- Black: interpolated point
- Red/yellow/green/blue: neighboring samples

Comparison



Nearest neighbor

Bi-linear

Bi-cubic

Key techniques in fully convolutional network

In-Network upsampling: “Unpooling”

Nearest Neighbor

1	2
3	4

Input: 2 x 2

1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Output: 4 x 4

“Bed of Nails”

1	2
3	4

Input: 2 x 2

1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Output: 4 x 4

In-Network upsampling: “Max Unpooling”

Max Pooling
Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4

5	6
7	8

Output: 2 x 2

. . .

Rest of the network

Max Unpooling
Use positions from pooling layer

1	2
3	4

Input: 2 x 2

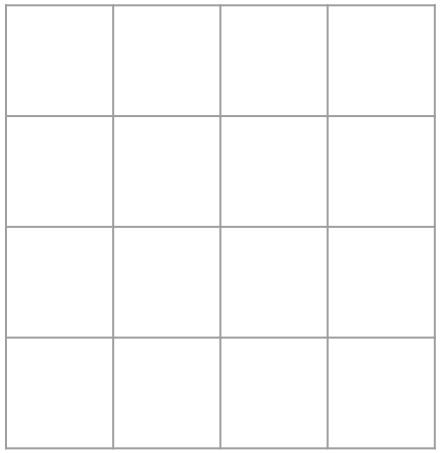
0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

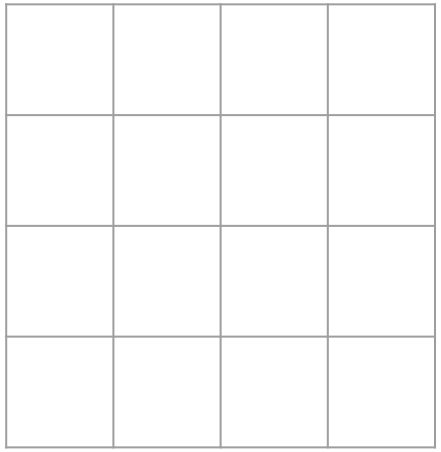
Corresponding pairs of downsampling and upsampling layers

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1



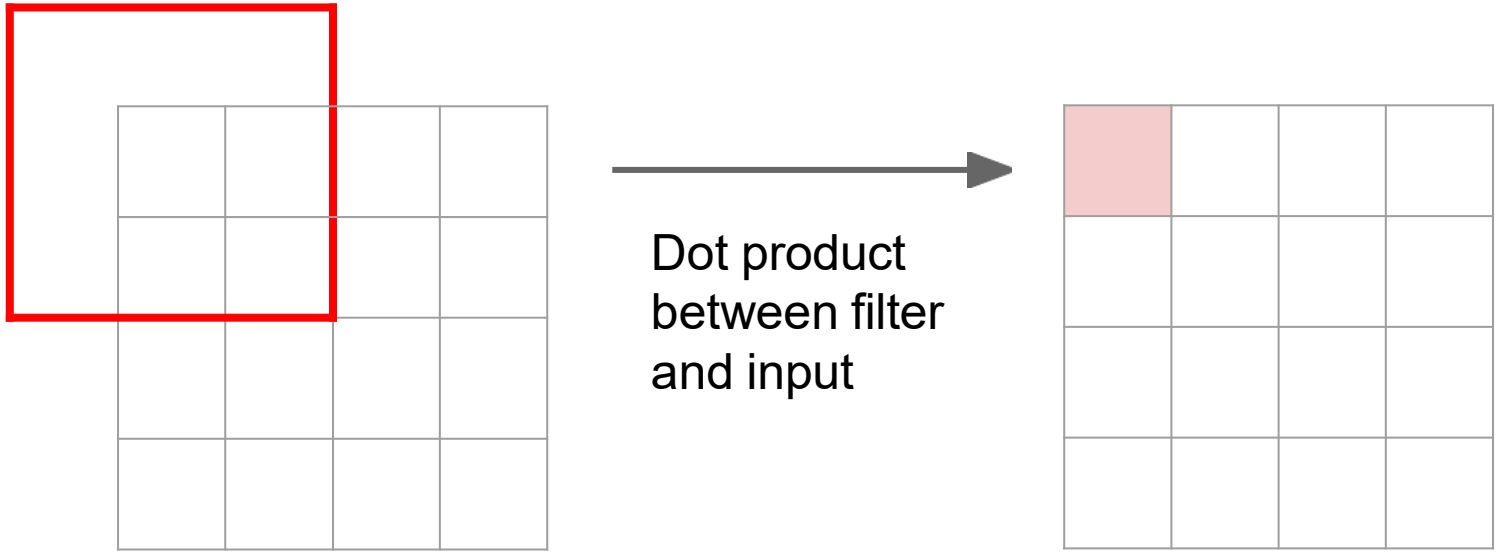
Input: 4 x 4



Output: 4 x 4

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1

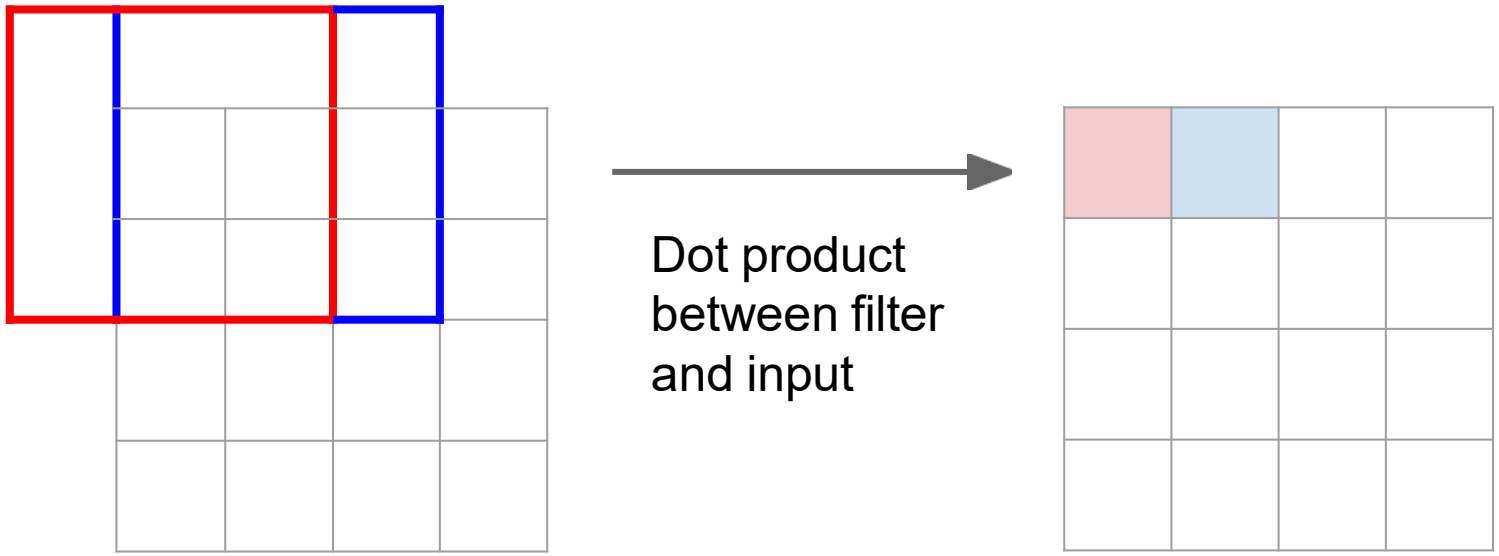


Input: 4 x 4

Output: 4 x 4

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1

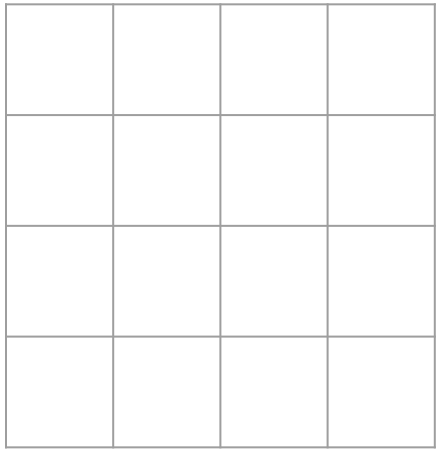


Input: 4 x 4

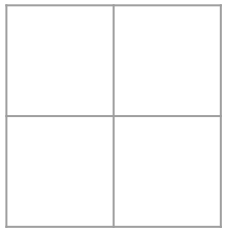
Output: 4 x 4

Learnable Upsampling

Recall: Normal 3 x 3 convolution, <u>stride 2</u> pad 1



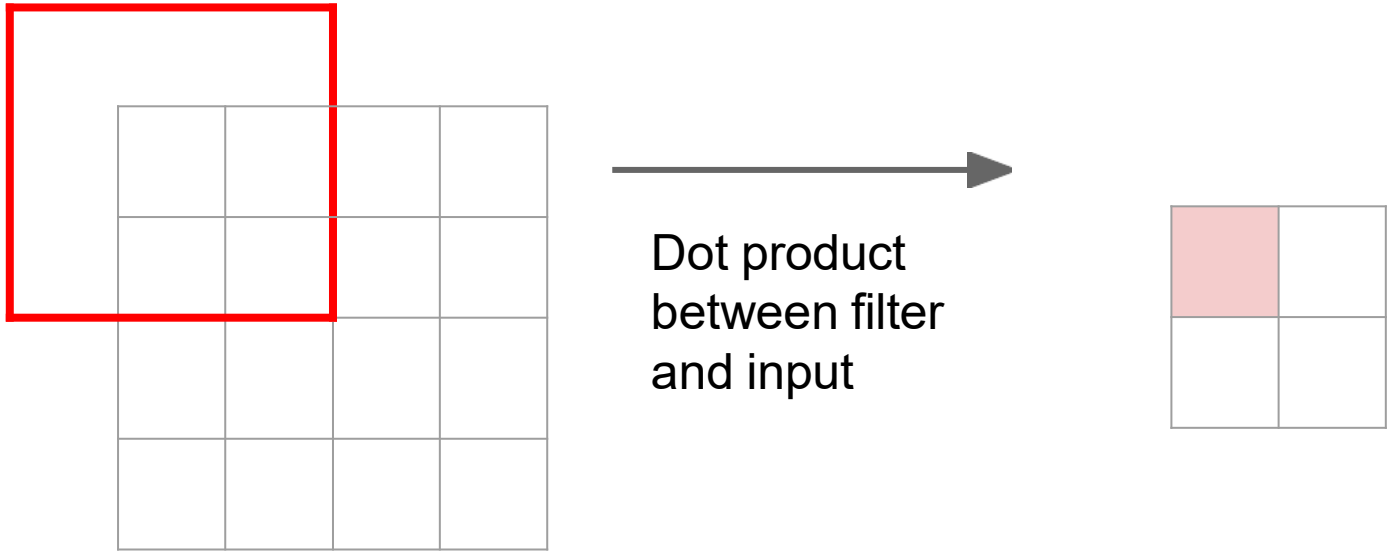
Input: 4 x 4



Output: 2 x 2

Learnable Upsampling

Recall: Normal 3 x 3 convolution, <u>stride 2</u> pad 1

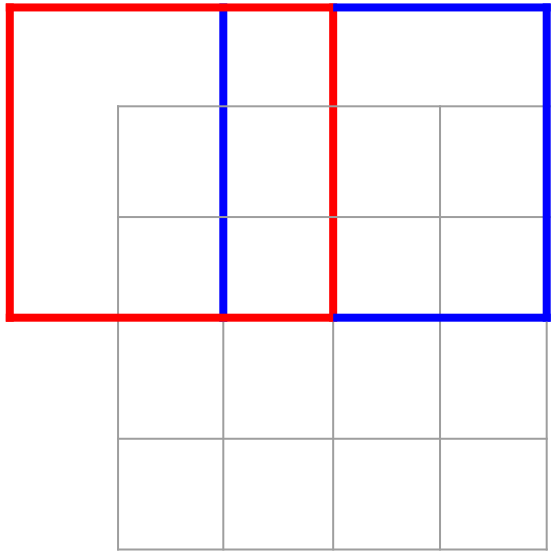


Input: 4 x 4

Output: 2 x 2

Learnable Upsampling

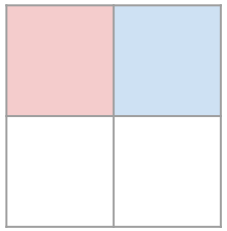
Recall: Normal 3 x 3 convolution, <u>stride 2</u> pad 1



Input: 4 x 4

Dot product between filter and input

Filter gives weight for input



Output: 2 x 2

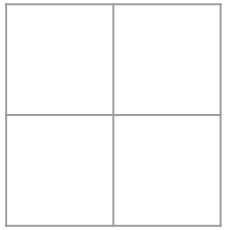
Filter moves 2 pixels in the input for every one pixel in the output

Stride gives ratio between movement in input and output

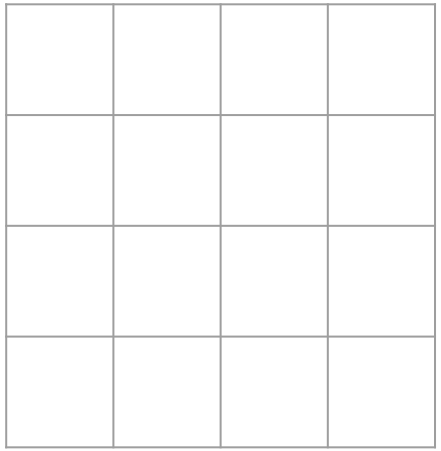
We can interpret strided convolution as “learnable downsampling”.

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1



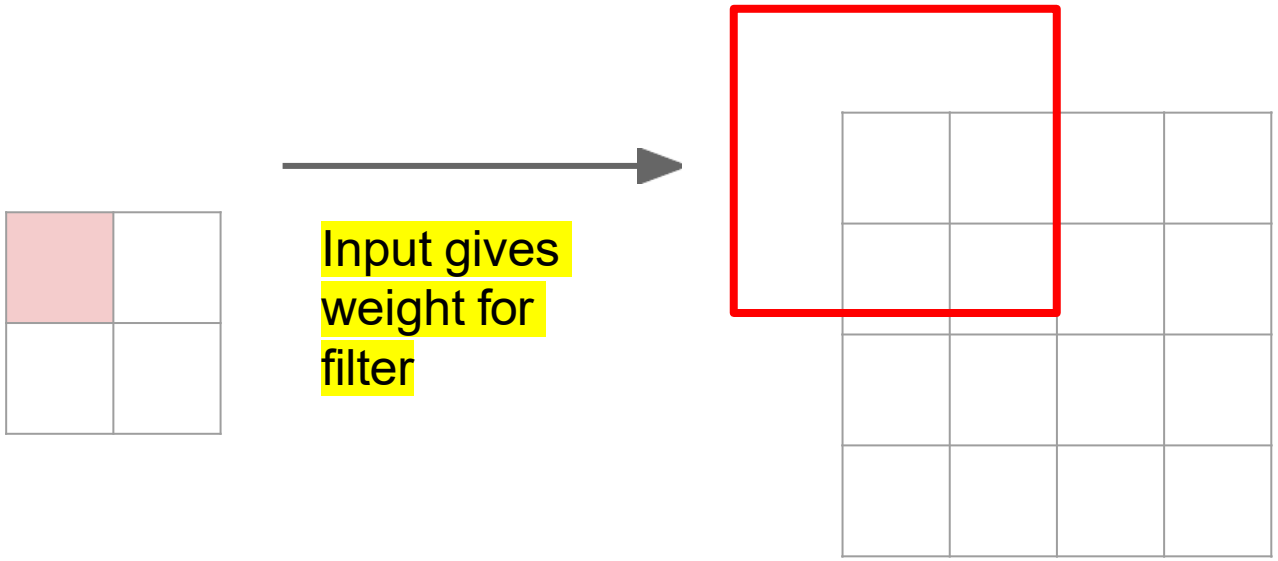
Input: 2 x 2



Output: 4 x 4

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1

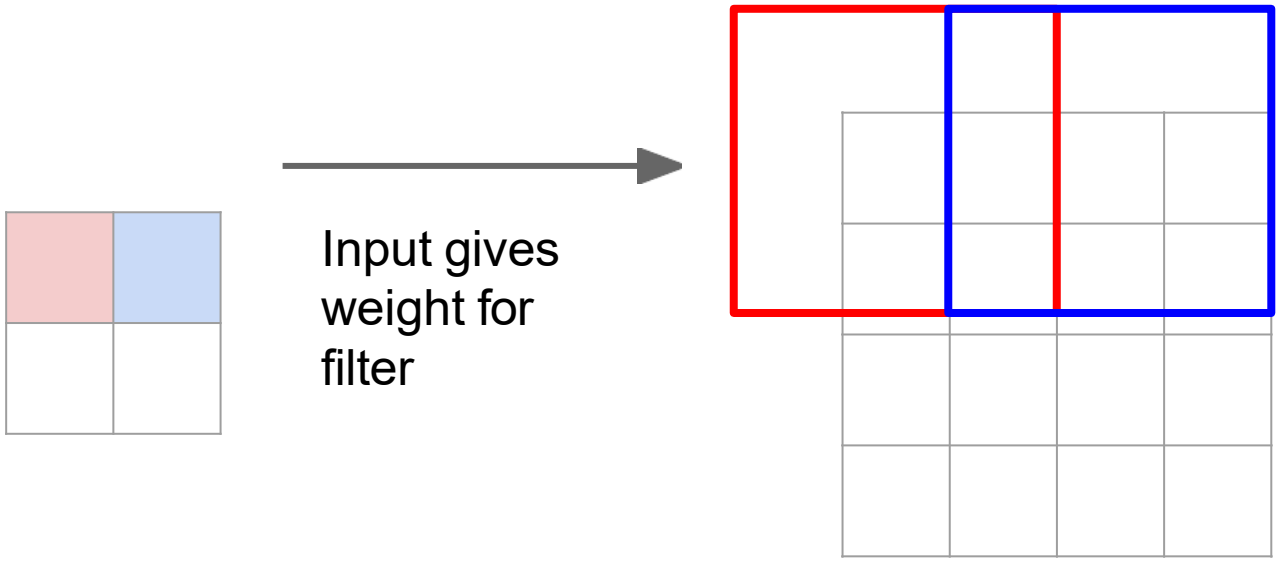


Input: 2 x 2

Output: 4 x 4

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1

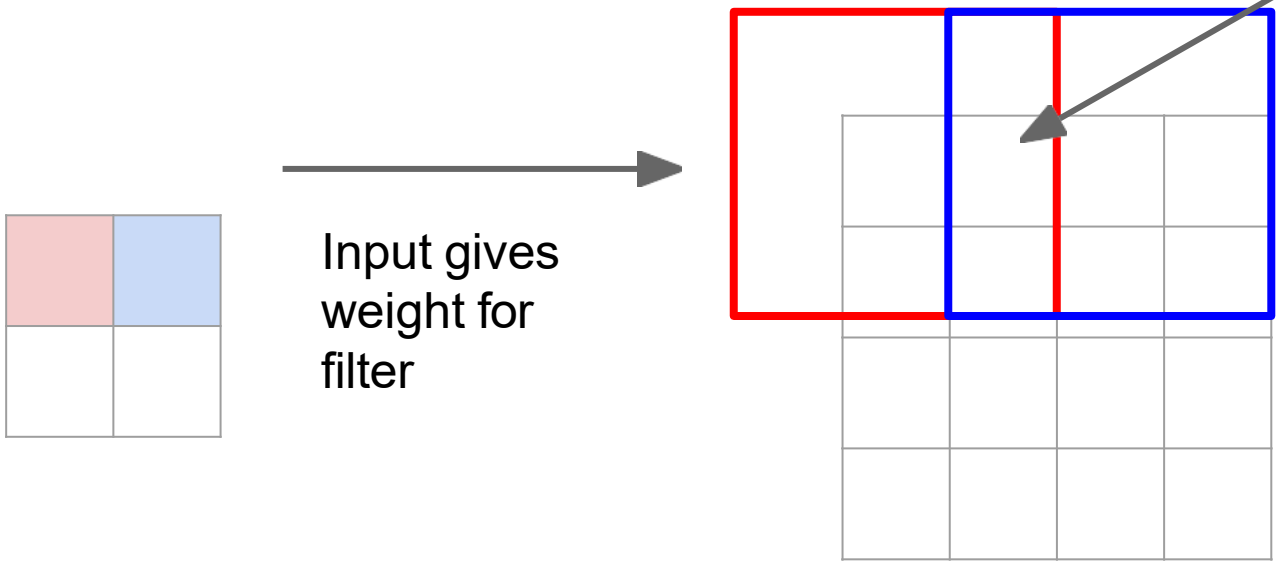


Filter moves 2 pixels in the <u>output</u> for every one pixel in the <u>input</u>

Stride gives ratio between movement in output and input

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1



Input gives weight for filter

Sum where output overlaps

Input: 2 x 2

Output: 4 x 4

Filter moves 2 pixels in the <u>output</u> for every one pixel in the <u>input</u>

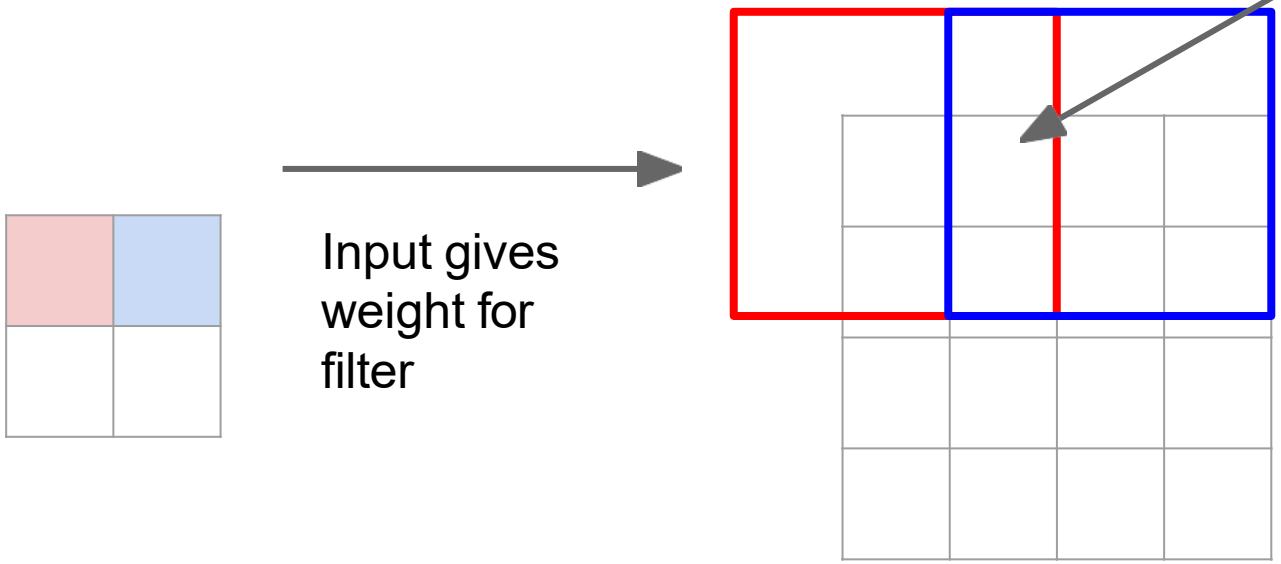
Stride gives ratio between movement in output and input

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1

Sum where output overlaps

Q: Why is it called transposed convolution?



Input gives weight for filter

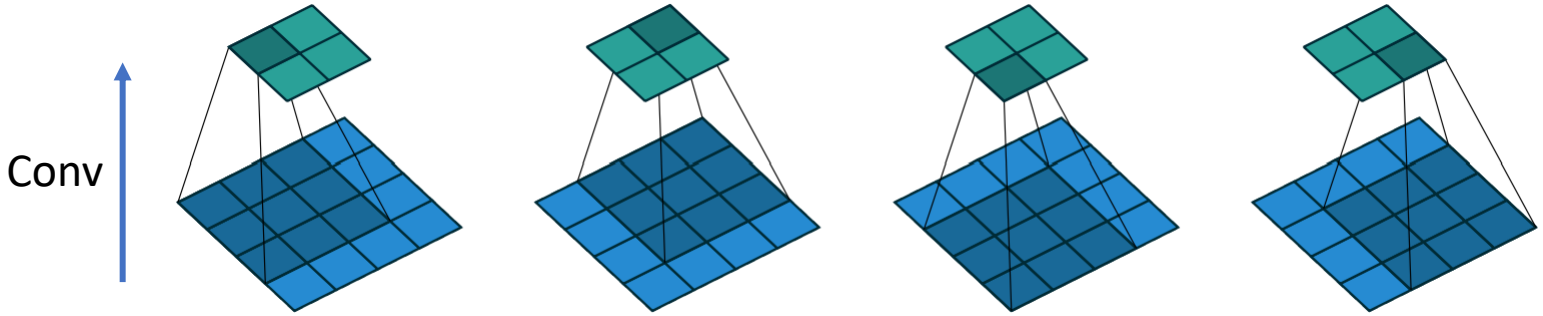
Input: 2 x 2

Output: 4 x 4

Filter moves 2 pixels in the <u>output</u> for every one pixel in the <u>input</u>

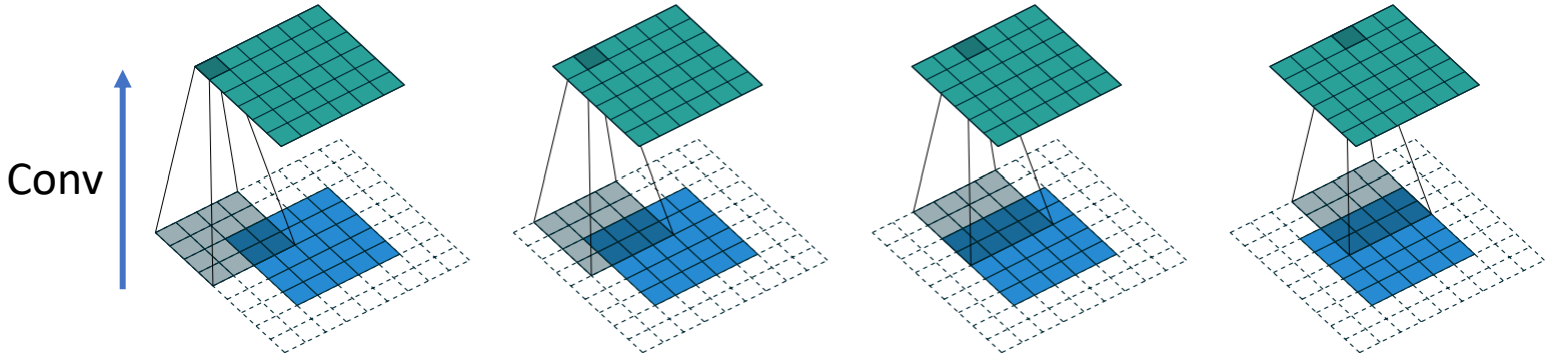
Stride gives ratio between movement in output and input

From Conv to Transposed Conv



Blue: input
Dark blue: kernel
Green: Output

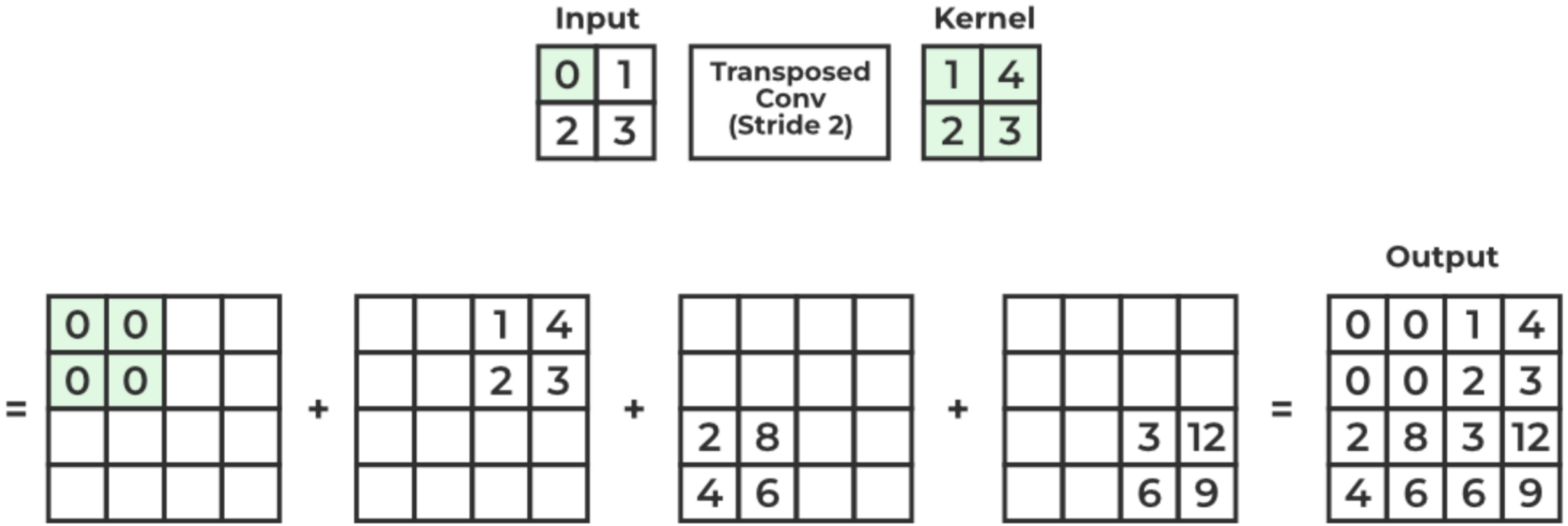
- Convolving a 3 × 3 kernel over a 4 × 4 input using unit strides (i.e., i = 4, k = 3, s = 1 and p = 0)



- Convolving a 4 × 4 kernel over a 5 × 5 input padded with a 2 × 2 border of zeros using unit strides (i.e., i = 5, k = 4, s = 1 and p = 2)

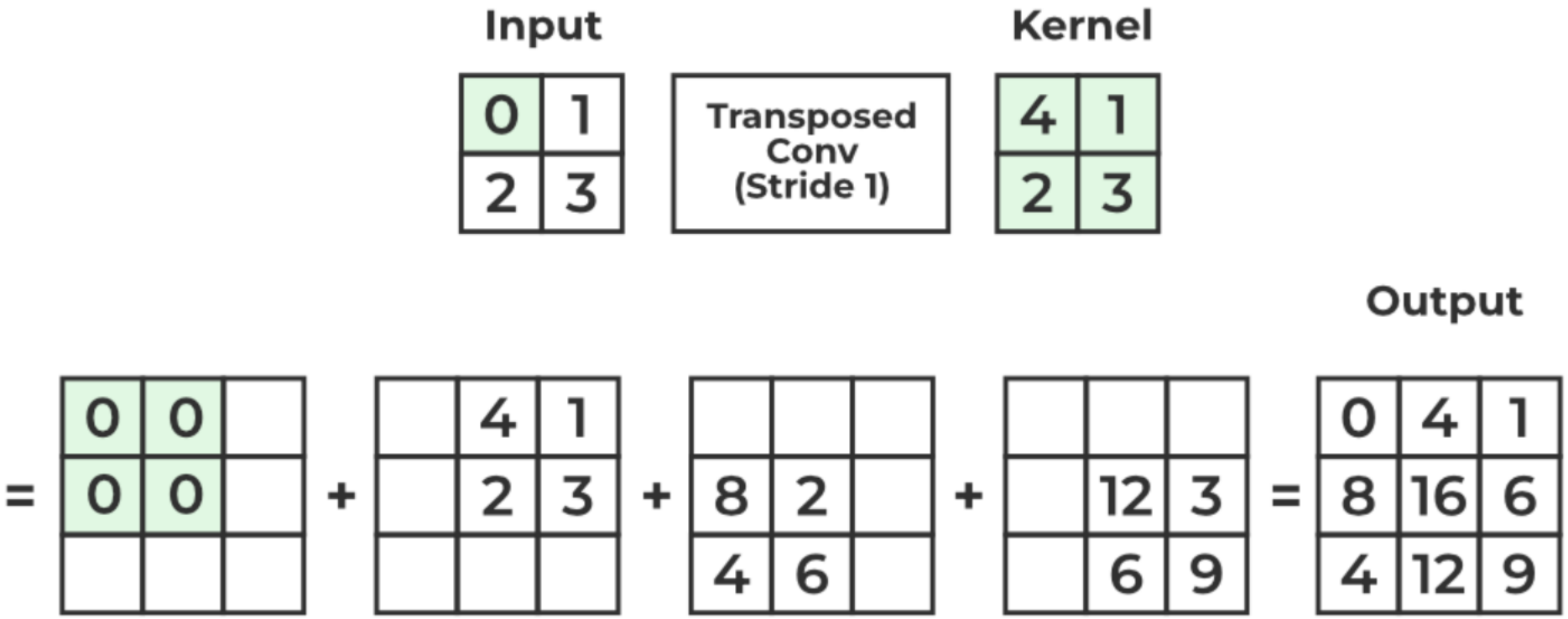
Transposed Conv

- Conv: Filter gives weight for input
- Transpose conv: Inputs give weight for filter
- Stride = 2



Transposed Conv

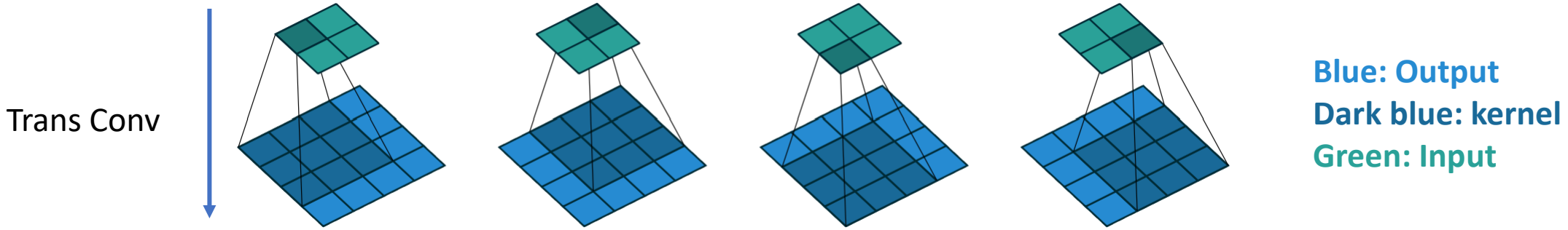
- Conv: Filter gives weight for input
- Transpose conv: Inputs give weight for filter
- Stride = 1



Shape of the output

- Image input size: I_h and I_w (height, width)
- Size of the kernel: K_h and K_w
- Stride: s_h (s_w), determines the step size for the input indices
- Padding: p, determines the number of pixels to add to the edges of the input before performing the convolution.
- Then the output of the transposed convolutional layer will be
  - $O_h = (I_h - 1) \times s_h + K_h - 2p$
  - $O_w = (I_w - 1) \times s_w + K_h - 2p$

From Conv to Transposed Conv



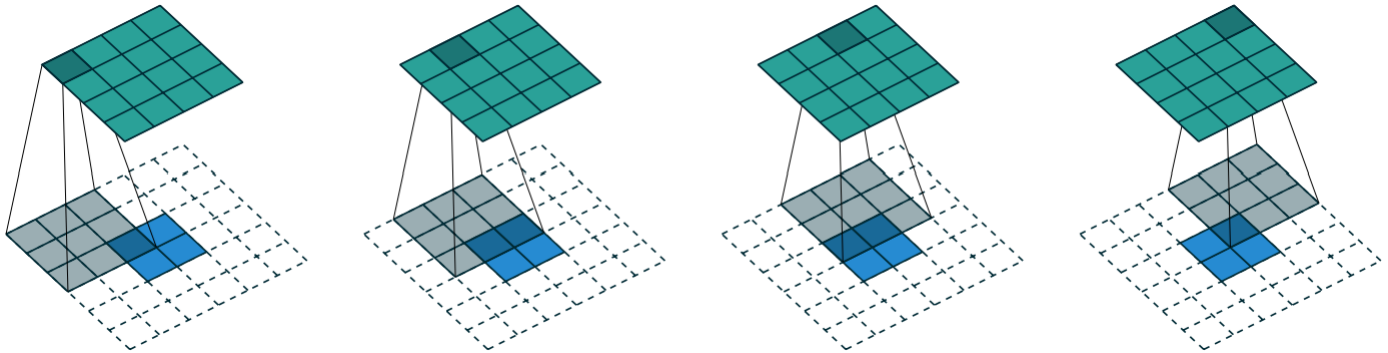
- Convolving a 3 × 3 kernel over a 4 × 4 input using unit strides (i.e., i = 4, k = 3, s = 1 and p = 0)
- Conv: Filter gives weight for input
- Transpose conv: Inputs give weight for filter (kernel)

- The transpose of this convolution will then have an output of shape 4 × 4 when applied on a 2 × 2 input.

$$O_h = (I_h - 1) \times s_h + K_h - 2p$$
$$O_w = (I_w - 1) \times s_w + K_h - 2p$$

From Conv to Transposed Conv

Trans Conv



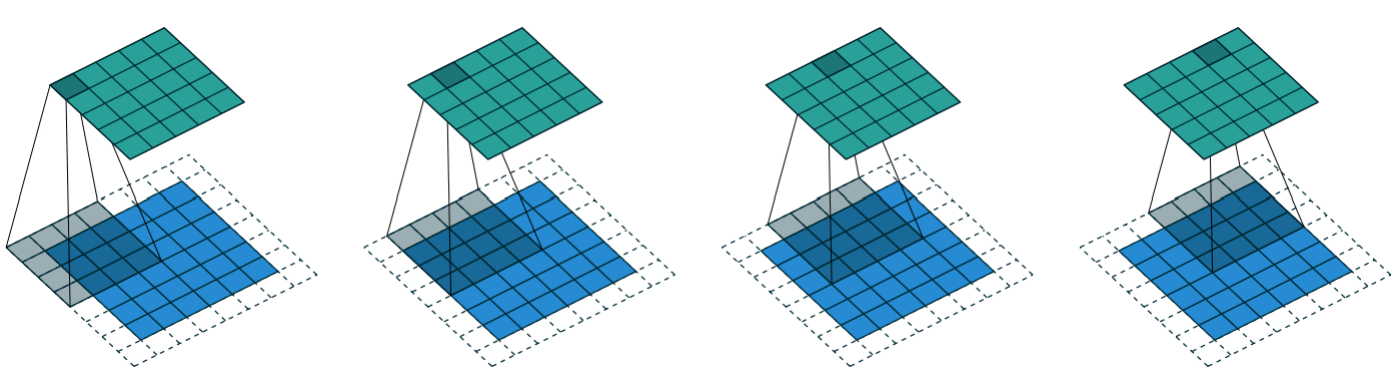
Blue: Output
Dark blue & Gray: kernel
Green: Input

- The transpose of convolving a 3 × 3 kernel over a 4 × 4 input using unit strides (i.e., i = 4, k = 3, s = 1 and p = 0)
- It is equivalent to convolving a 3 × 3 kernel over a 2 × 2 input padded with a 2 × 2 border of zeros using unit strides (i.e., i'= 2, k'= k, s'= 1 and p'= 2).

$$O_h = (I_h - 1) \times s_h + K_h - 2p$$
$$O_w = (I_w - 1) \times s_w + K_h - 2p$$

From Conv to Transposed Conv

Trans Conv



Blue: Output
Dark blue & Gray: kernel
Green: Input

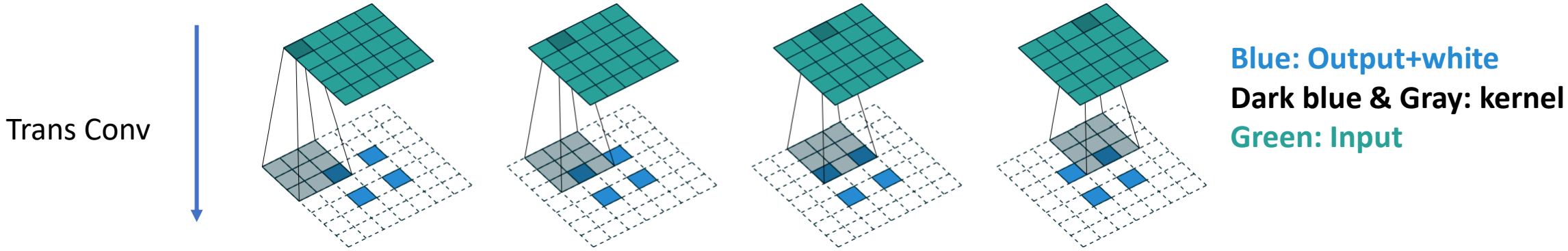
- The transpose of convolving a 4 × 4 kernel over a 5 × 5 input with a 2x2 border of zeros using unit strides (i.e., i = 5, k = 4, s = 1 and p = 2)
- It is equivalent to convolving a 4 × 4 kernel over a 6 × 6 input padded with a 1 × 1 border of zeros using unit strides (i.e., i'= 6, k'= k, s'= 1 and p'= 1).

$$O_h = (I_h - 1) \times s_h + K_h - 2p$$
$$O_w = (I_w - 1) \times s_w + K_h - 2p$$

Thinking…

- Does the kernel size need to be larger than the input in order to increase the resolution?
- How should we efficiently design a kernel for transposed convolution?

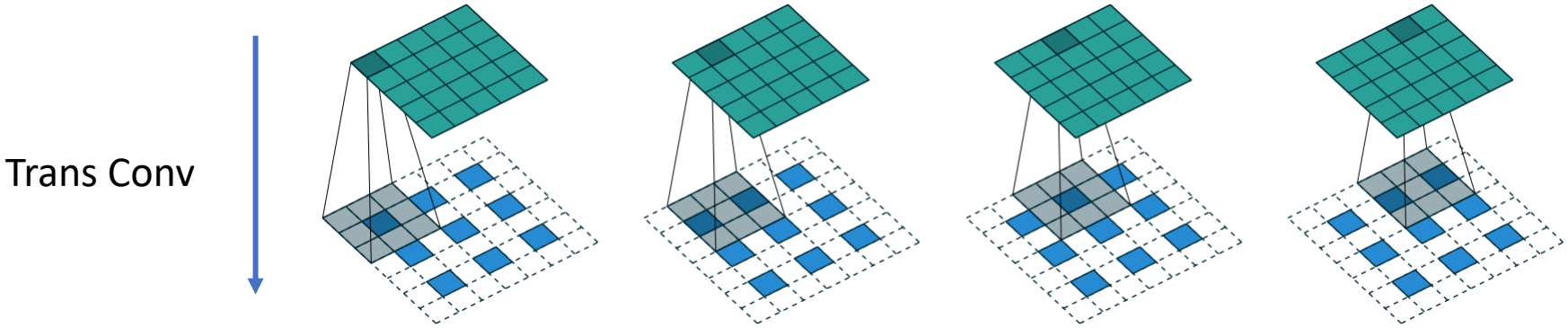
Adding zeros near input



- The transpose of convolving a 3 × 3 kernel over a 5 × 5 input using 2 × 2 strides (i.e., i = 5, k = 3, s = 2 and p = 0)
- It is equivalent to convolving a 3 × 3 kernel over a 2 × 2 input (with 1 zero inserted between inputs) padded with a 2 × 2 border of zeros using unit strides.

$$O_h = (I_h - 1) \times s_h + K_h - 2p$$
$$O_w = (I_w - 1) \times s_w + K_h - 2p$$

Adding zeros near input



Blue: Output+white
Dark blue & Gray: kernel
Green: Input

- The transpose of convolving a 3×3 kernel over a 5×5 input padded with a 1 × 1 border of zeros using 2 × 2 strides (i.e., i = 5, k = 3, s = 2 and p = 1).
- It is equivalent to convolving a 3 × 3 kernel over a 2 × 2 input (with 1 zero inserted between inputs) padded with a 1 × 1 border of zeros using unit strides

$$O_h = (I_h - 1) \times s_h + K_h - 2p$$
$$O_w = (I_w - 1) \times s_w + K_h - 2p +$$

https://arxiv.org/pdf/1603.07285v1

Why and how to increase the resolution by 2?

- Why?
  - Think about Max Pooling

- How?
  - Typical setting: K=4, S=2, P=1
  - $O_h = (I_h - 1) \times s_h + K_h - 2p$
  - If $O_h = 2I_h$, $s_h = 2$, then $2I_h = 2I_h - 2 + K_h - 2p$
  - Thus $K_h = 2p + 2$

Morden Image Segmentation Neural Network

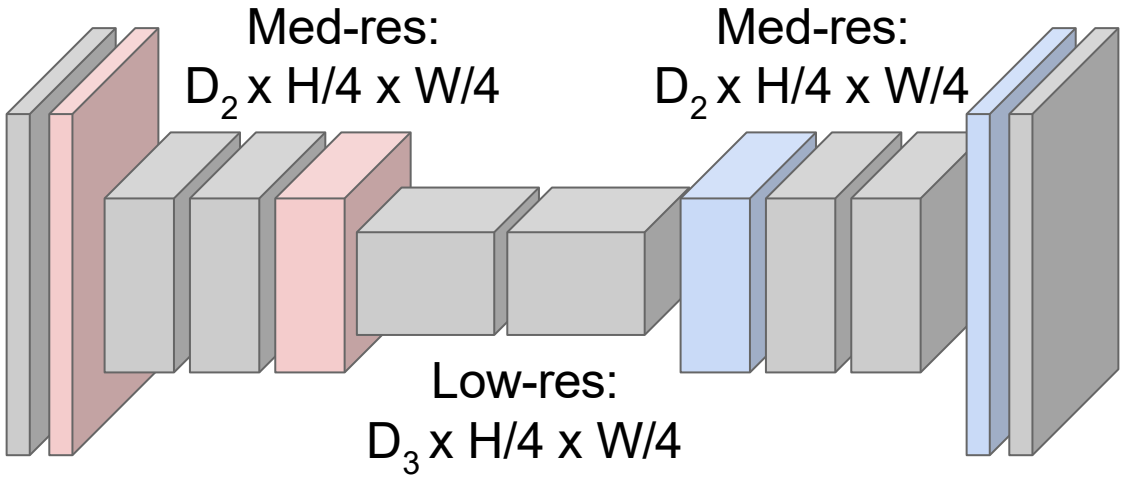
Semantic Segmentation Idea: Fully Convolutional

Downsampling: Pooling, strided convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling: Unpooling or strided transposed convolution

Input:
3 x H x W



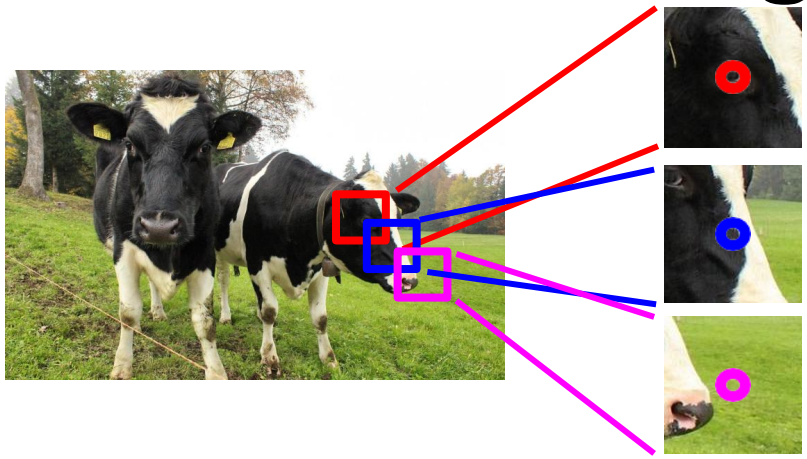
High-res:
D_1 x H/2 x W/2

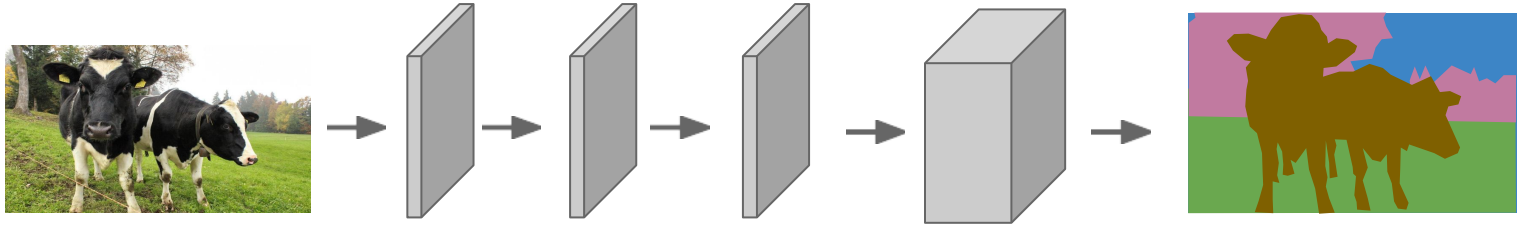
High-res:
D_1 x H/2 x W/2

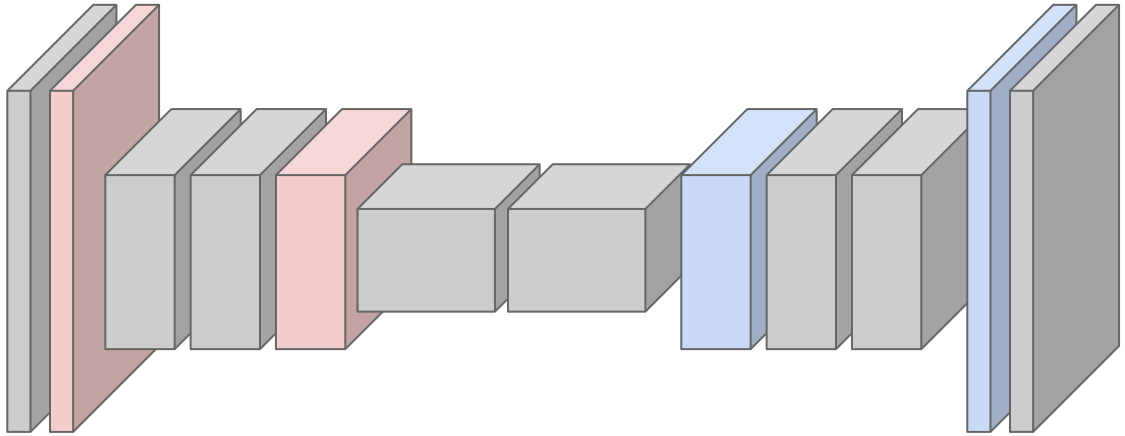
Predictions:
H x W

Long, Shelhamer, and Darrell, “Fully Convolutional Networks for Semantic Segmentation”, CVPR 2015
Noh et al, “Learning Deconvolution Network for Semantic Segmentation”, ICCV 2015

Semantic Segmentation: Summary



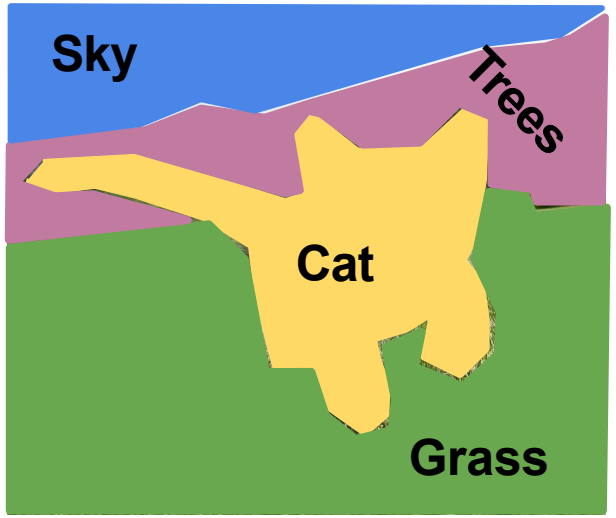




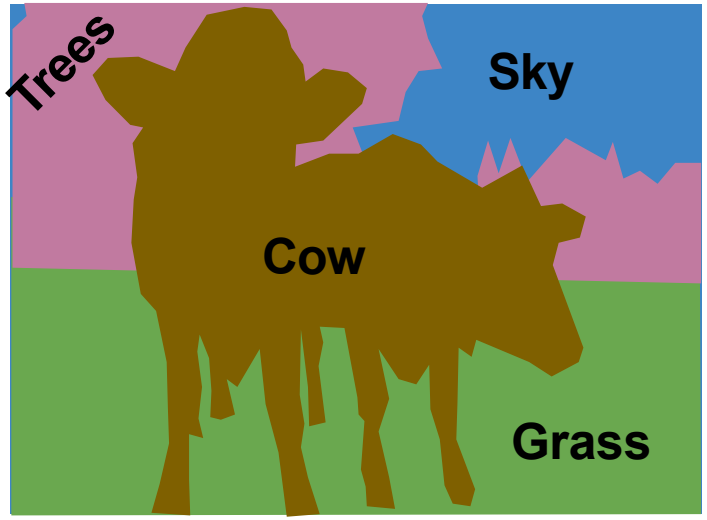
Semantic Segmentation

Label each pixel in the image with a category label

Don’t differentiate instances, only care about pixels

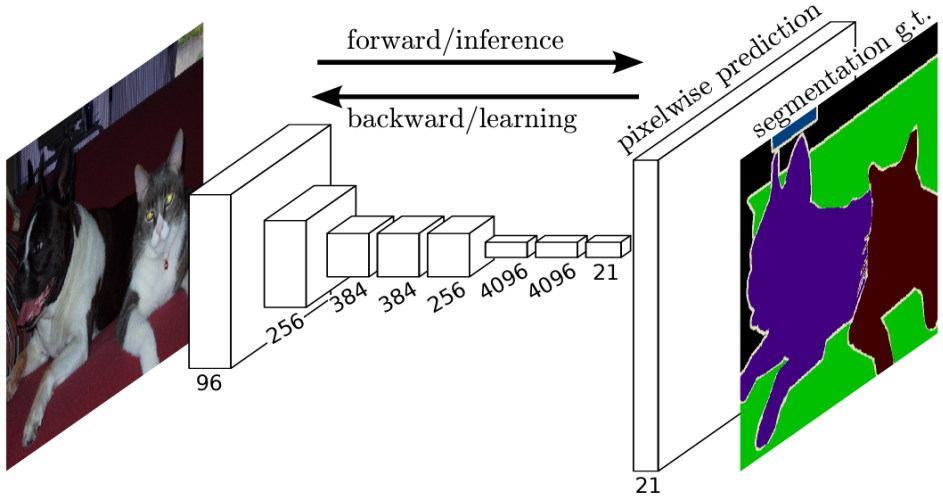


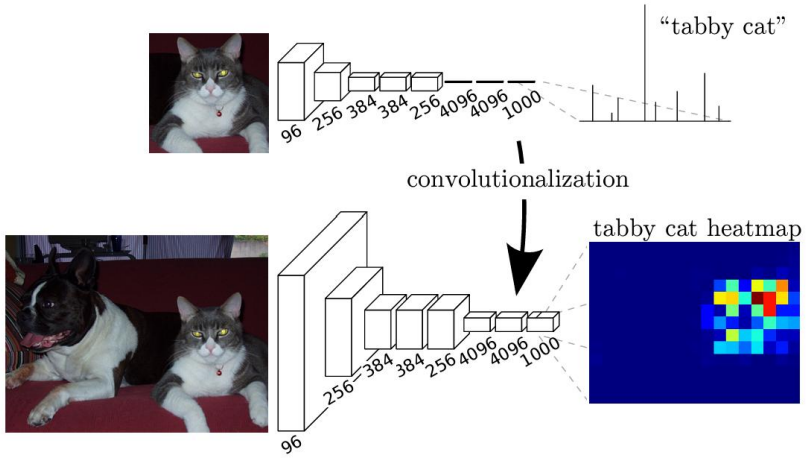
This image is CC0 public domain



Fully convolutional networks (FCN)

- No Fully Connected Layers: replace the fully connected layers with convolutional layers, which allows them to handle input images of any size and produce a corresponding output map that is the same size as the input.
- Pixel-wise Predictions: FCNs output a map (or a dense prediction) for each pixel.



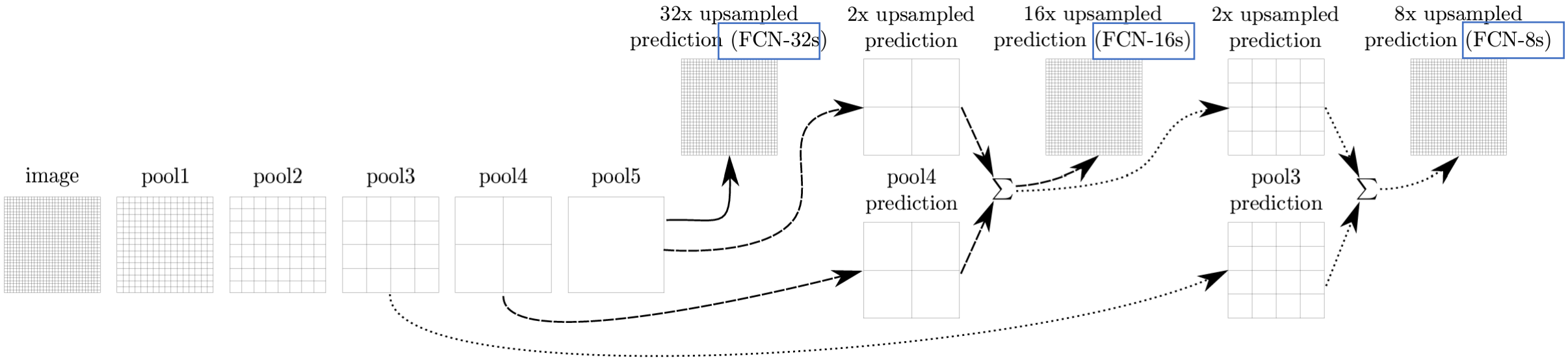


Long et al. https://arxiv.org/abs/1411.4038

Fully convolutional networks (FCN)

- Deconvolution: increase the spatial resolution of the output map (upsample)
- Skip Connections: Skip connections combine low-level features (e.g., edges, textures) from earlier layers with high-level features (e.g., object parts) from deeper layers.

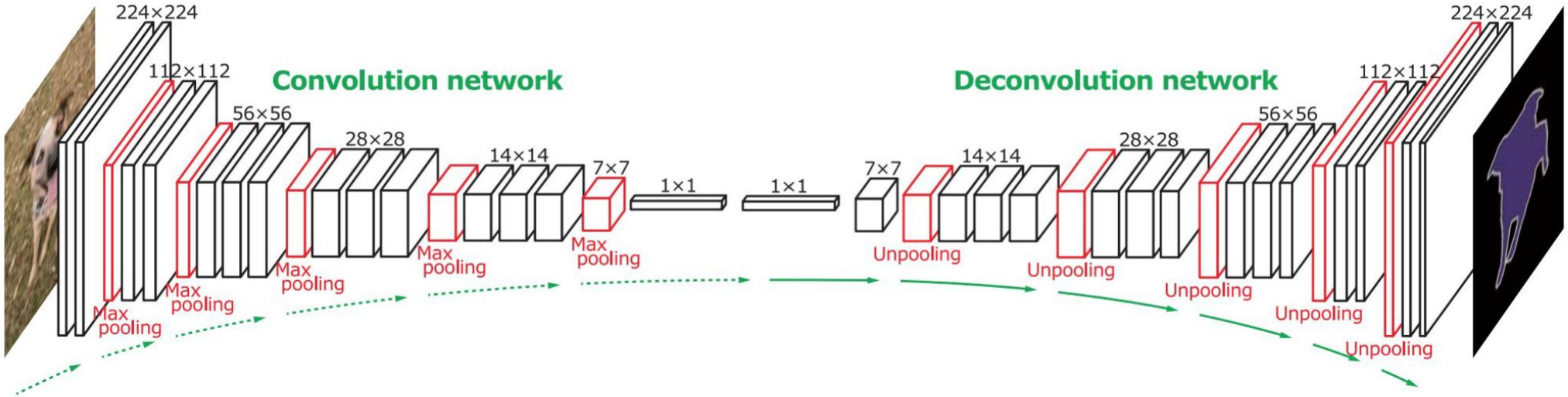
The **number** (8, 16, 32) refers to the **downsampling factor** in the final convolutional layer



Long et al. https://arxiv.org/abs/1411.4038

Encoder-Decoder Based Models

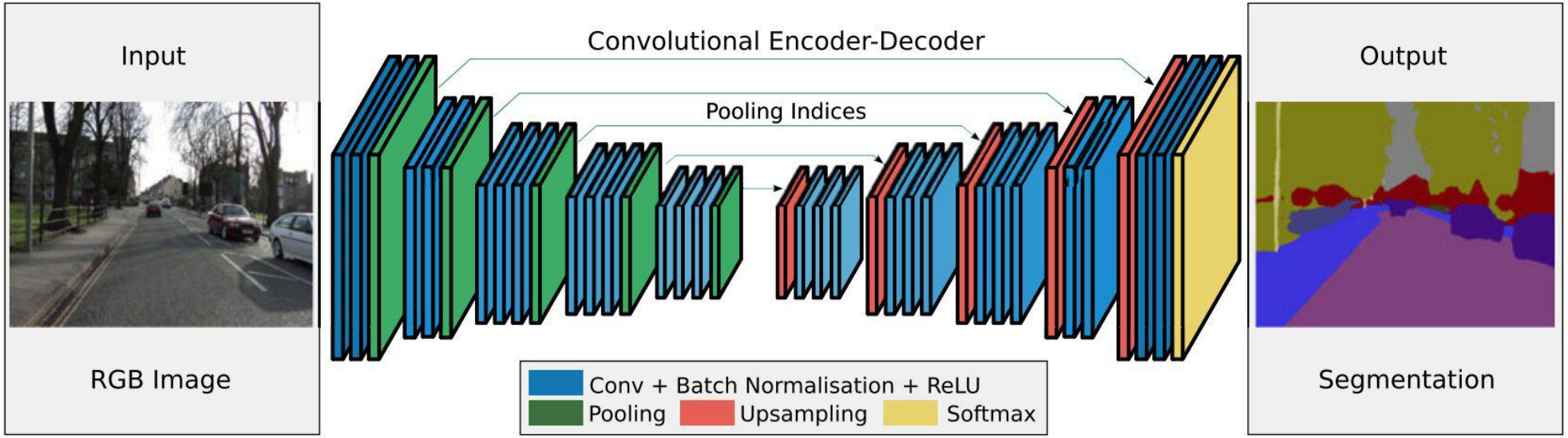
- The Encoder processes the input image and compresses it into a compact representation that captures essential features
- The Decoder takes the compact representation generated by the Encoder and expands it into an image with labeled pixels



H. Noh, S. Hong, and B. Han, “Learning deconvolution network for semantic segmentation,” in Proceedings of the IEEE international conference on computer vision, 2015, pp. 1520–1528.

Encoder-Decoder Based Models- SegNet

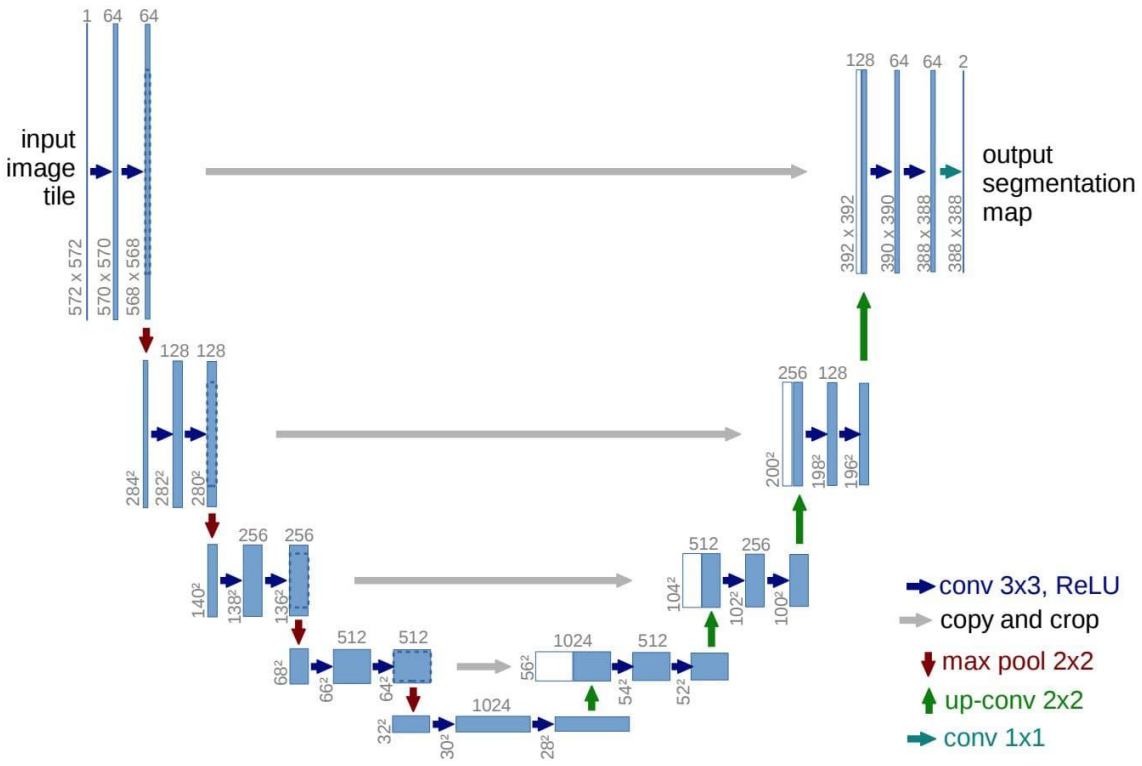
- Use pooling indices computed in the max-pooling step of the corresponding encoder to perform non-linear upsampling.
- This eliminates the need for learning to up-sample



H. Noh, S. Hong, and B. Han, "Learning deconvolution network for semantic segmentation," in Proceedings of the IEEE international conference on computer vision, 2015, pp. 1520–1528.

Encoder-Decoder Based Models- UNet

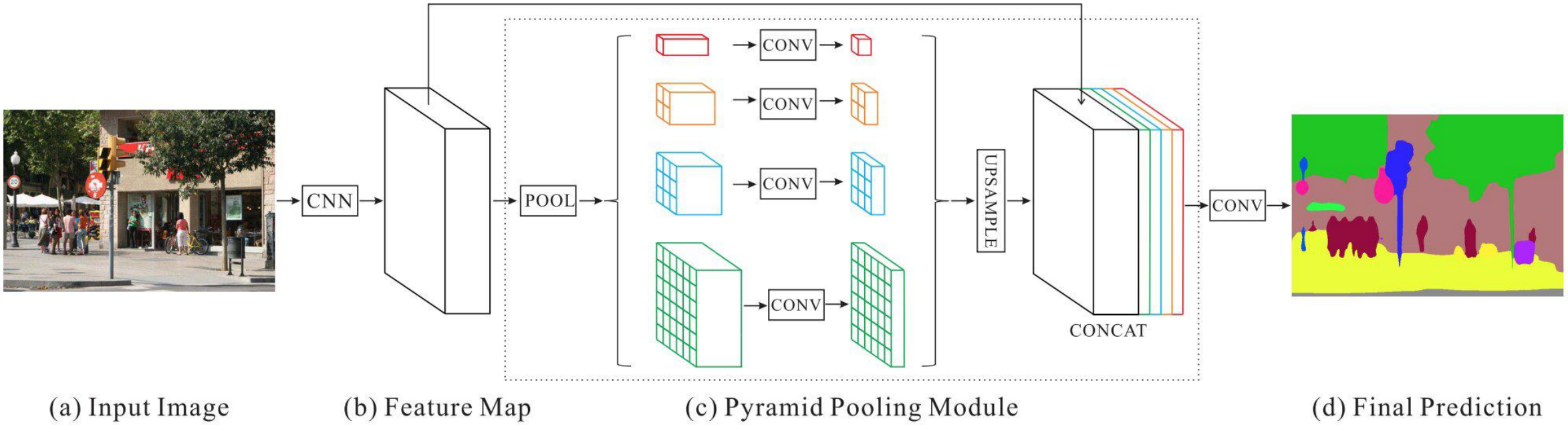
- The U-Net architecture (Figure 14) comprises two parts, a contracting path to capture context, and a symmetric expanding path that enables precise localization
- Skip-connection: maintain high-resolution



Ronneberger, O., Fischer, P. and Brox, T., 2015. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18* (pp. 234-241). Springer International Publishing.

Multi-scale and Pyramid Network based models

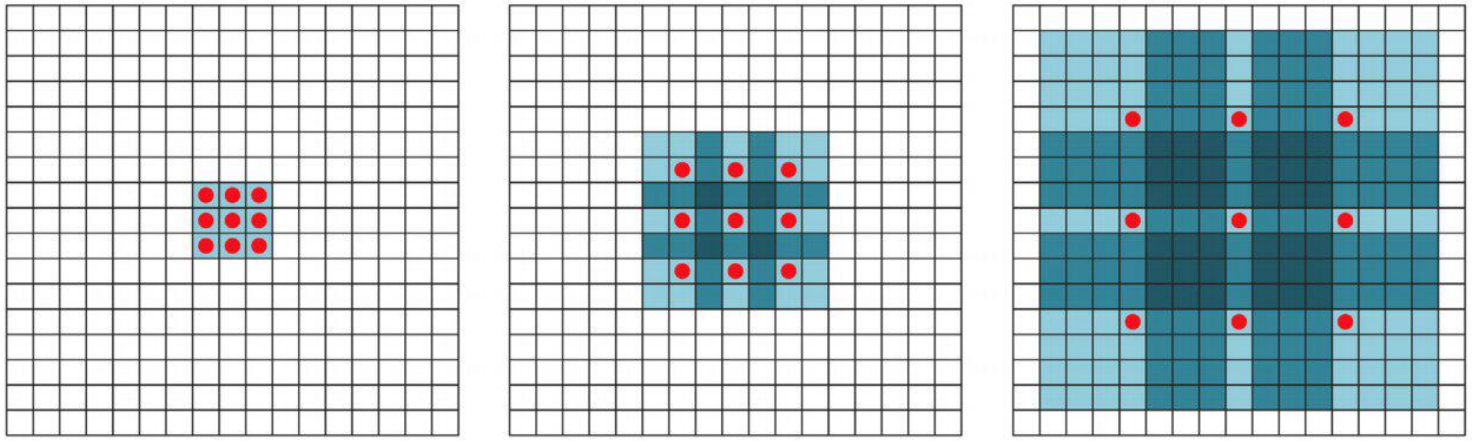
- Feature Pyramid Network (FPN)/Pyramid Scene Parsing Network (PSPN)
- The inherent multi-scale, pyramidal hierarchy of deep CNNs was used to construct feature



H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, “Pyramid scene parsing network,” in Proceedings of the IEEE conference on computer vision and pattern recognition, 2017, pp. 2881–2890.

Dilated Convolutional Models and DeepLab Family

- Dilated convolution (a.k.a. “atrous” convolution) introduces another parameter to convolutional layers, the dilation rate
- For example, a 3x3 kernel with a dilation rate of 2 will have the same size receptive field as a 5x5 kernel while using only 9 parameters, thus enlarging the receptive field with no increase in computational cost

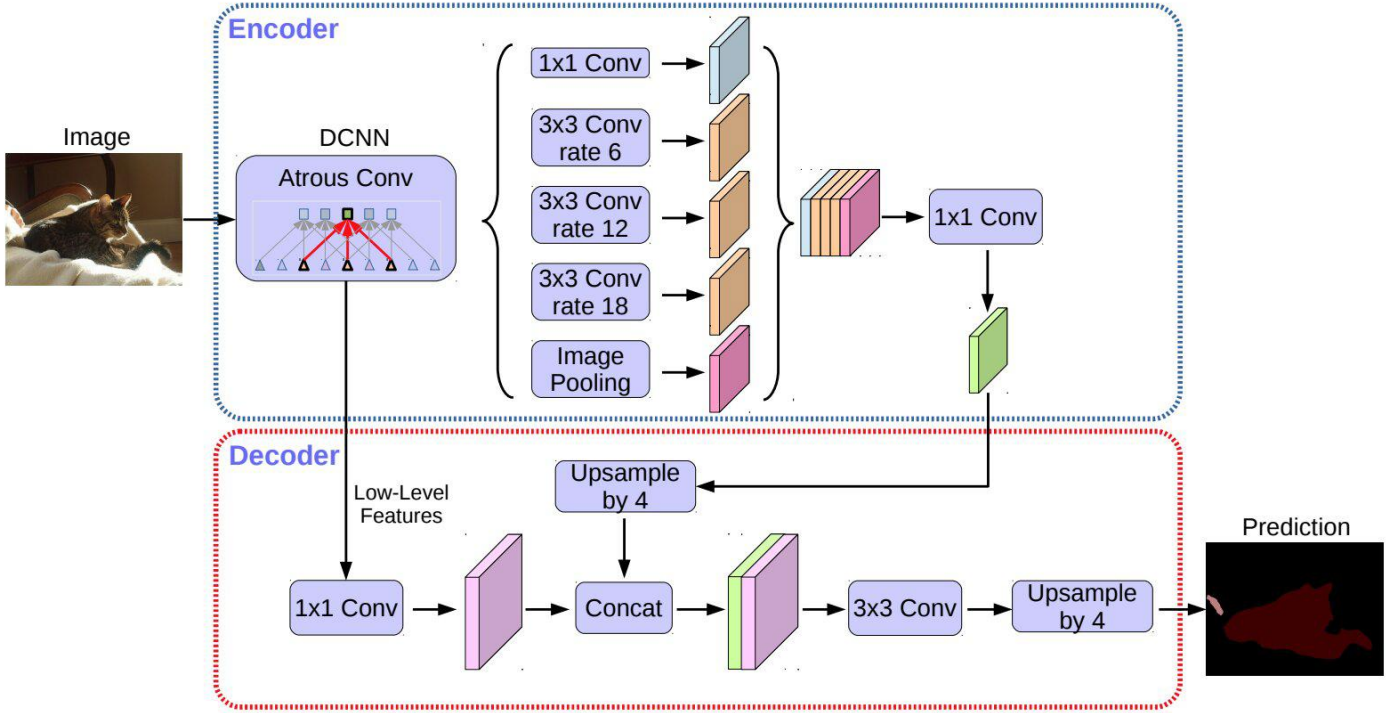


(a) 1-dilated (b) 2-dilated (c) 3-dilated

L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, “Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs,” IEEE transactions on pattern analysis and machine intelligence, vol. 40, no. 4, pp. 834–848, 2017

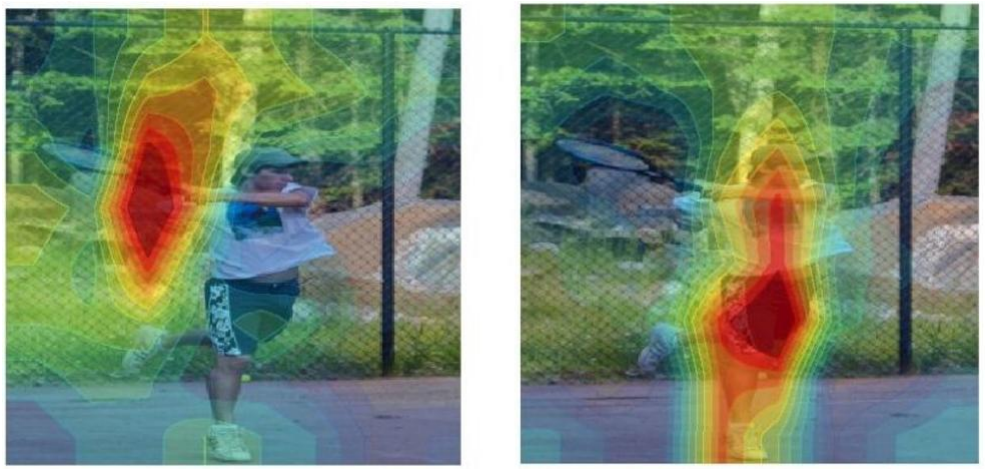
Dilated Convolutional Models and DeepLab Family

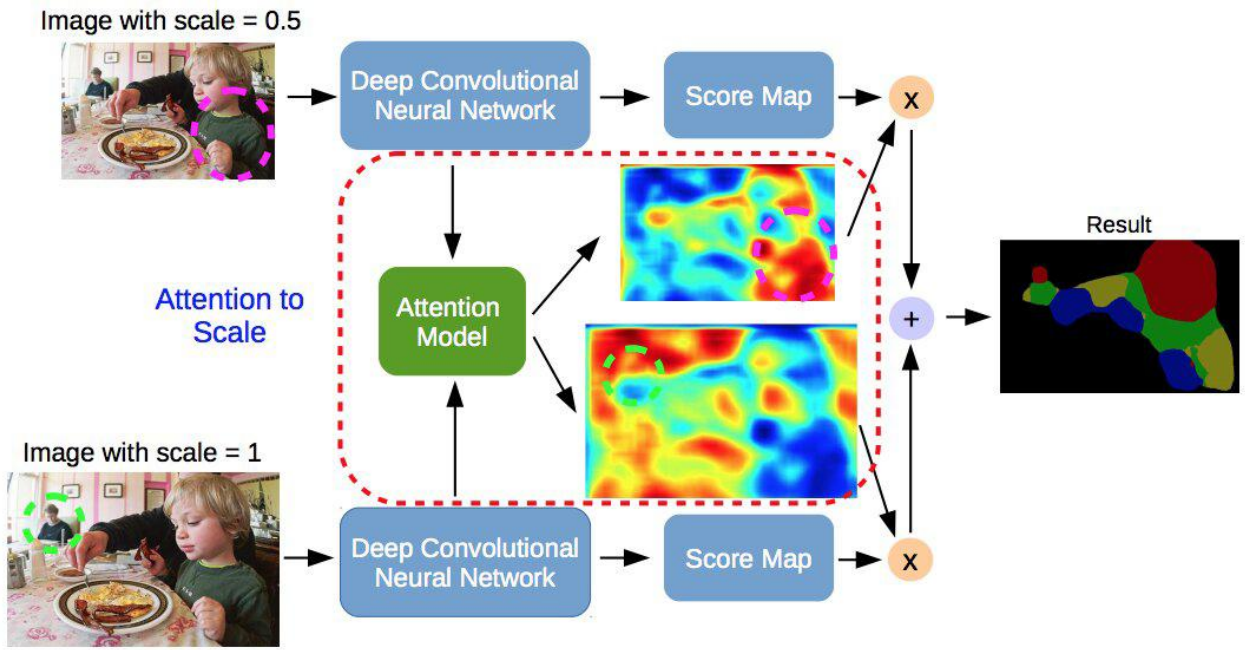
- Deeplabv3+, which uses an encoder-decoder architecture, including atrous separable convolution, composed of
- a depthwise convolution (spatial convolution for each channel of the input) and
- pointwise convolution (1x1 convolution with the depthwise convolution as input).



L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, “Encoder-decoder with atrous separable convolution for semantic image segmentation,” in Proceedings of the European conference on computer vision (ECCV), 2018, pp. 801–818.

Attention based models





- Attention-based semantic segmentation model.
- The attention model learns to assign different weights to objects of different scales; e.g., the model assigns
- large weights on the small person (green dashed circle) for features from scale 1.0,
- and large weights on the large child (magenta dashed circle) for features from scale 0.5

Summary

- Computer Vision and Image Segmentation
- Semantic Segmentation: Sliding window/convolution/fully convolution
- Key techniques in fully convolutional network
  - Upsampling/unpooling
  - Convolution transpose
- Morden Image Segmentation Neural Network
  - Fully convolution network
  - Encoder-decoder based network
  - Multi-scale and Pyramid Network
  - Dilated convolutional models and deeplab family
  - Attention based models